Physics-Informed Neural Networks for Additive Manufacturing: 1D Stefan Problem

EML 5360: Additive Manufacturing

Goutam Kumar Biswas Maddox Gonzalez

April 27, 2026

Abstract

Accurate thermal prediction is central to additive manufacturing because the temperature field governs melt-pool evolution, solidification, and ultimately part quality. This paper reimplements a 1D Stefan benchmark in PyTorch using a physics-informed neural network (PINN) built on the enthalpy form of transient heat conduction, which captures phase change without explicitly tracking the solid-liquid interface. Two boundary-condition strategies are examined: a hard formulation that embeds the Dirichlet values directly into the network output and a soft formulation that enforces the same values through penalty terms in the loss. The hard-BC model pre-trains to initial-condition errors near 10^{-8} and remains more stable throughout optimization, whereas the soft-BC model is markedly more sensitive to initialization and can collapse to an unphysical near-uniform temperature field unless additional scaling and normalization are introduced. Against the FEM reference field, the hard-BC PINN reaches a relative L_2 error of about 1.1×10^{-3} at $N_x = 50$ and changes little as the collocation count increases to 40,000 points (0.9×10^{-3} at $N_x = 200$). Over the same range, FEM shows the expected improvement with mesh refinement, starting at 3.6×10^{-2} and reaching 2.3×10^{-3} at $N_x = 200$. Taken together, these results indicate that hard boundary enforcement is the more reliable choice for this benchmark. They also show that, for this 1D problem, increasing collocation density alone does not materially improve PINN accuracy once a moderate sample count has already been reached.

1 Introduction

Accurate thermal modeling is one of the main bottlenecks in additive manufacturing (AM). As a laser or electron beam deposits material, it creates highly localized heating, steep temperature gradients, and rapid solidification. Those thermal histories directly influence melt-pool shape, residual stress, and final microstructure, so predicting the temperature field is not simply a numerical exercise; it is tied to part quality and process control.

The difficulty is that AM thermal problems are strongly coupled and highly transient. Material properties change with temperature, latent heat must be accounted for during melting and solidification, and the effective phase boundary moves continuously through the domain. Finite element methods (FEM) can resolve these effects well, but doing so requires repeated spatial discretization, careful treatment of moving interfaces, and increasing computational cost as the problem grows in geometric and physical complexity.

Physics-informed neural networks (PINNs) offer a different route. Rather than solving the governing equations on a mesh, a PINN represents the solution with a neural network and trains that network so that its predictions satisfy the underlying partial differential equation, the boundary conditions, and the initial condition. Since the framework introduced by Raissi, Perdikaris, and Karniadakis [5], PINNs have attracted attention across heat transfer and mechanics because they combine data-driven approximation with explicit physical constraints.

In the AM setting, Zhu, Liu, and Yan [9] showed that a PINN could reproduce a 1D Stefan benchmark consisting of a graphite substrate and an aluminum deposit separated by a phase-change region. That problem provides a useful test bed because it retains the essential thermal physics of AM while remaining simple enough to study systematically. The present work reimplements that benchmark in PyTorch and uses it to answer two practical questions that matter for reliable PINN deployment: how strongly does boundary-condition enforcement strategy affect optimization stability, and how much does accuracy change when the collocation density is increased?

These questions are not minor implementation details. Boundary conditions are often one of the hardest parts of PINN training, and the literature still does not offer a universal rule for when hard structural enforcement should be preferred over soft penalty-based enforcement [1]. Likewise, collocation count controls both computational effort and how well the model samples the space-time domain, which becomes especially important if the method is to be extended from this 1D benchmark to realistic 2D or 3D AM problems.

This paper therefore focuses not only on final error values, but also on how the model trains, where it fails, and what design choices make it behave more reliably. In practice, two nontrivial optimization pathologies appeared during implementation, and documenting those issues turned out to be as informative as reporting the final benchmark numbers.

2 Related Work

The present study sits at the intersection of two established lines of work: enthalpy-based phase-change modeling and physics-informed neural networks. On the numerical side, Voller, Cross, and Markatos [6] showed that latent heat can be absorbed into an enthalpy formulation, turning a sharp free-boundary problem into a smoother energy-balance problem that can be solved without explicitly tracking the solid-liquid interface. That idea remains one of the standard formulations for solidification and melting, and it is the governing framework adopted in the benchmark considered here.

On the machine-learning side, Raissi et al. [5] introduced PINNs as a way to solve forward and inverse PDE problems by minimizing a physics-based residual loss through automatic differentiation. The appeal of the approach is clear: because the governing equation is enforced at scattered collocation points, the method avoids a conventional mesh and can in principle approximate the solution over the full space-time domain with a single neural network.

That promise, however, comes with optimization challenges. Wang et al [8] showed that different components of the PINN loss can generate gradients with vastly different magnitudes, which means some constraints may dominate while others are effectively ignored during training. In heat-transfer applications, Cai et al. [1] similarly noted that steep thermal gradients and competing objective terms are among the most common sources of instability. These observations are directly relevant to the current problem, where the network must satisfy a variable-coefficient PDE, boundary constraints, and an initial condition simultaneously.

Free-boundary problems make the situation even more delicate. Wang et al [7] demonstrated that PINNs can represent moving interfaces by treating interface position as an additional trainable quantity. The present benchmark uses a different but closely related strategy: instead of representing a sharp interface explicitly, it replaces the solid-liquid jump with a 20 K mushy zone. That smoothing simplifies the problem numerically while preserving the main phase-change physics.

Within additive manufacturing, Zhu et al. [9] were among the first to apply PINNs to thermal prediction with phase change, using the same 1D bi-material Stefan problem reimplemented in this paper. More recent work has pushed the idea further. Peng and Panesar [4] extended thermal PINNs to multi-layer directed energy deposition, where repeated heating and cooling cycles create a far richer thermal history than a single-pass benchmark. Madir et al. [3] addressed one of the key limitations of standard PINNs by concentrating collocation points near the phase boundary, which improved accuracy in the stiffest region of the domain without a proportional increase in total samples.

The broader picture, summarized by Farrag et al. [2], is that physics-informed machine learning for AM is still developing. Most published studies remain confined to low-dimensional or simplified geometries, direct experimental validation is uncommon, and uncertainty quantification is rarely reported. In that context, a careful reexamination of a 1D benchmark remains useful, especially when the emphasis is placed not only on reproducing final errors but also on understanding training stability and model behavior.

3 Methodology

This section is organized in two parts. The first and primary part presents the PINN formulation, since the neural-network model is the main subject of the study. The second part summarizes the FEM reference solver used to generate the initial-condition data, validate the predicted fields, and build the refinement-study baseline. Keeping the two parts separate makes it easier to distinguish what is learned by the PINN from what is supplied by the numerical reference.

3.1 PINN Methodology

3.1.1 Benchmark Setup and Governing Physics

The benchmark domain is a one-dimensional rod spanning $x \in [-0.4, 0.4]\,\mathrm{m}$. The graphite substrate occupies the left half ($x \leq 0$), while the aluminum deposit occupies the right half ($x > 0$). The simulation window is $t \in [5.0, 10.0]\,\mathrm{s}$, which represents the thermal response after

deposition in the original benchmark. Constant Dirichlet temperatures are imposed at both ends of the rod, $T(-0.4,t) = 298.15\,\mathrm{K}$ and $T(0.4,t) = 973.15\,\mathrm{K}$. The initial temperature profile fed to the PINN is sampled from the fine-grid FEM reference field described in Appendix A. In the baseline hard/soft comparison, $N_0 = 150$ initial-condition points are sampled from that field without replacement; in the resolution study, this supervised set is increased to $N_0 = 300$. In other words, the PINN is not asked to infer the initial thermal state from scratch; it is anchored to a reference profile and then trained to recover the full space-time field while respecting the governing physics.

The same material properties are used in both the PINN and FEM models and are summarized in Table 1. Graphite is treated as a single-phase solid with constant properties, whereas aluminum is allowed to transition between solid and liquid states through a mushy zone bounded by the solidus temperature $T_s = 913.15\,\mathrm{K}$ and the liquidus temperature $T_l = 933.15\,\mathrm{K}$. Latent heat is included through an enthalpy-based formulation so that phase change is captured without explicitly tracking a sharp moving interface.

Table 1: Material properties used in the simulation.

Material	ρ (kg/m^3)	κ (W/m K)	c_p (J/kg K)	L (kJ/kg)
Aluminum (liquid)	2555	91	1190	398
Aluminum (solid)	2555	211	1190	398
Graphite	2200	100	1700	—

To describe the thermal evolution, the model uses the enthalpy form of transient heat conduction [6]:

$$\rho c_p \frac{\partial T}{\partial t} + \rho L \frac{\partial f_L}{\partial t} = \frac{\partial}{\partial x}\left(\kappa \frac{\partial T}{\partial x}\right), \tag{1}$$

where T is temperature, f_L is the liquid volume fraction, ρ is density, c_p is specific heat, L is latent heat, and κ is thermal conductivity.

The phase state is introduced through a clipped linear mushy-zone model:

$$f_L(T) = \mathrm{clip}\left(\frac{T - T_s}{T_l - T_s},\, 0,\, 1\right). \tag{2}$$

This definition lets the liquid fraction vary smoothly from 0 to 1 across the 20 K transition interval. In the PINN implementation, graphite keeps constant material coefficients, while aluminum conductivity varies with the local liquid fraction. As a result, the PDE residual is evaluated with temperature-dependent coefficients at every collocation point rather than with a simplified constant-property approximation. That point is important, because much of the difficulty of the problem comes from the fact that the physics changes with the local thermal state.

3.1.2 Network Architecture

The PINN maps the space-time coordinates (x, t) directly to a temperature prediction T_θ. The architecture is

$$[2,\ 200,\ 200,\ 200,\ 200,\ 1], \tag{3}$$

corresponding to four fully connected hidden layers with 200 neurons each, tanh activations in the hidden layers, and a linear output layer. In total, the model contains roughly 123,000

trainable parameters. Xavier normal initialization is used for the weights, and the inputs are normalized to the range $[-1,1]$ before entering the first layer so that the early activations remain in a regime where tanh does not saturate too aggressively.

This input normalization is more than a cosmetic preprocessing step. The two inputs, space and time, have different physical units and different numerical ranges, so feeding them directly into the network would bias the first layer toward whichever variable has the larger scale. Mapping both coordinates to a common dimensionless interval keeps the activations centered, improves gradient flow through the tanh layers, and makes the optimizer respond to changes in x and t on a more balanced footing. For a PINN, this is especially important because the model is differentiated repeatedly with respect to both inputs when forming the PDE residual.

The original implementation by Zhu et al. [9] employed a Swish (SiLU) activation. In the present reimplementation, tanh was chosen instead because it behaved more consistently across repeated runs and avoided the occasional overflow issues observed with Swish when intermediate values became too large during early optimization. The change does not alter the overall PINN formulation, but it does make training noticeably more stable, which is why it was adopted for the experiments reported here.

Figure 1 shows the network topology.

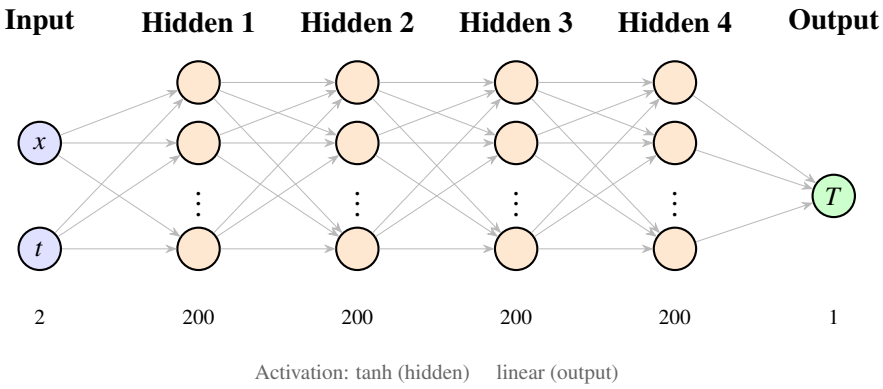


Figure 1: PINN architecture used in all experiments. The network takes the space-time coordinates (x,t) as input, passes them through four hidden layers of 200 neurons with tanh activation, and returns a scalar temperature prediction T. Dots indicate additional neurons omitted from the diagram for clarity.

3.1.3 Boundary-Condition Enforcement

Hard Boundary Condition Enforcement In the hard-BC variant, the Dirichlet boundary values are built directly into the network output, so the model satisfies them by construction regardless of the learned weights. A smooth Heaviside-like mask $H(x)$ is used to suppress the trainable correction at the boundaries while allowing the network to act in the interior, with softness parameter $\varepsilon = 10^{-3}$:

$$T_\theta(x,t) = T_{bc}(x)\big(1 - H(x)\big) + \hat{T}_{raw}(x,t)\,H(x), \tag{4}$$

where $T_{bc}(x)$ is a linear interpolant between the prescribed endpoint temperatures and $\hat{T}_{raw}$ is the unconstrained network output. This construction reduces the training objective to the initial-condition mismatch and the PDE residual, which removes one source of competition among loss terms and, as shown later, makes the optimization process much more stable.

Soft Boundary Condition Enforcement In the soft-BC variant, the network is allowed to predict arbitrary boundary values and is guided toward the correct ones through penalty terms in the loss. To keep the raw output in a physically meaningful range, the final prediction is passed through an affine transformation:

$$T_\theta(x,t) = T_{ref} + T_{scale} \cdot \hat{h}(x,t), \tag{5}$$

where $T_{ref} = 635.65\,\mathrm{K}$ is the midpoint of the boundary temperatures, $T_{scale} = 337.5\,\mathrm{K}$ is half of their range, and $\hat{h}$ is the raw linear network output. This scaling step proved essential for keeping the soft-BC model away from the unphysical saddle-point solution discussed in Section 4.5.

During the main training phase, the soft-BC formulation also augments the standard collocation set with $3N_f$ additional points clustered near the expected phase boundary. In the implementation, those extra points are sampled inside $x \in [-0.05, 0.05]$ so that the model sees the stiffest thermal region more densely. That refinement is intentionally omitted during pretraining so that the network first learns the global temperature structure without being pulled toward a narrow interface band too early. This small detail turned out to matter, because overemphasizing the interface before the global field is learned makes the soft-BC model easier to destabilize.

3.1.4 Normalization, Loss Functions, Training, and Collocation Sampling

Normalization is used at several levels in the PINN because the problem mixes quantities with very different numerical scales. In the soft-BC formulation, the network output is written in the affine form of Eq. (5), which centers predictions near the middle of the physical temperature range and scales them by half of that range. The logic is straightforward: starting from random weights, it is much easier for the network to learn an $O(1)$ correction around a physically reasonable temperature level than to generate temperatures near 300–1000 K directly from an unconstrained raw output. This scaling also helps prevent the model from drifting toward the near-uniform saddle-point solution identified later in Section 4.5.

The loss function combines the physics residual with the available data-based constraints. At each collocation point (x_k, t_k), the residual associated with Eq. (1) is computed through automatic differentiation and then normalized by $T_{NRM} = T_{scale}^2 \approx 113{,}906\,\mathrm{K}^2$:

$$r_k = \frac{1}{T_{NRM}} \left[\rho c_p \frac{\partial T_\theta}{\partial t} + \rho L \frac{\partial f_L}{\partial t} - \frac{\partial}{\partial x}\left(\kappa \frac{\partial T_\theta}{\partial x} \right) \right]_{(x_k,t_k)}. \tag{6}$$

For the hard-BC formulation, the training objective is

$$\mathscr{L}_{hard} = \frac{1}{N_0} \sum_{i=1}^{N_0} \left| T_\theta(x_i^0, t_0) - T_i^0 \right|^2 + \lambda_f \frac{1}{N_f} \sum_{k=1}^{N_f} |r_k|^2, \tag{7}$$

with $\lambda_f = 10^{-8}$. In other words, the hard-BC model balances only the initial-condition fit and the PDE residual because the boundary values are already satisfied analytically. The soft-BC loss adds two boundary mismatch terms:

$$\mathscr{L}_{soft} = \mathscr{L}_{hard} + \frac{1}{N_b} \sum_{j=1}^{N_b} \left| T_\theta(-0.4, t_j) - 298.15 \right|^2 + \frac{1}{N_b} \sum_{j=1}^{N_b} \left| T_\theta(0.4, t_j) - 973.15 \right|^2, \tag{8}$$

with $N_b = 150$ boundary time points drawn uniformly from $[t_0, t_f]$. This additional supervision makes the soft-BC objective more flexible, but it also introduces another competing gradient source, which makes the optimization problem harder to balance.

The normalization by T_{NRM} is used for the same reason. The temperature errors in the initial and boundary terms are measured in K^2, while the PDE residual combines time derivatives, spatial derivatives, and temperature dependent coefficients. Without scaling, one part of the loss can dominate the others purely because of its units and magnitude rather than because it is more important physically. Dividing the temperature-based losses and the residual by the same characteristic temperature scale keeps the optimization problem better conditioned and makes the weighting parameter λ_f meaningful rather than arbitrary.

Training follows a two-stage protocol. In the first stage, the network is pre-trained against the initial temperature profile interpolated from the FEM reference field onto the collocation x-locations. This warm start is important because it moves the network away from arbitrary random predictions before the full PDE residual is activated. Once that initial profile has been learned, the second stage turns on the full physics-informed loss and asks the network to satisfy the PDE throughout the space-time domain.

The justification for pre-training is tied directly to the structure of the problem. At random initialization, the network does not represent a meaningful temperature field, so the derived quantities that depend on temperature, such as the liquid fraction and the conductivity inside the mushy zone, are also poorly initialized. If the full PINN objective is enforced from the first iteration, the optimizer must simultaneously discover the approximate thermal field, satisfy the boundary and initial conditions, and reduce the PDE residual, all while the material coefficients are changing with the current prediction. Pre-training breaks that task into stages. By first fitting the initial temperature profile, the model begins the physics-constrained phase from a physically plausible state, with sensible thermal gradients and a much better guess for the phase distribution. In practice, this greatly improves stability, reduces the risk of collapse to an unphysical low-gradient solution, and gives the subsequent Adam and L-BFGS-B stages a more useful starting point.

In the baseline configuration, both models use 50,000 Adam iterations followed by 50,000 L-BFGS-B iterations during pre-training. The main training stage again uses 50,000 Adam iterations followed by 50,000 L-BFGS-B iterations. The hard-BC model is comparatively stable under this schedule, whereas the soft-BC model requires a smaller main-stage Adam learning rate (5×10^{-4}) together with gradient clipping at 1.0 to avoid large parameter jumps. The physics weight is set to $\lambda_f = 10^{-8}$, and all temperature-based mean-squared-error terms are normalized by $T_{NRM} = 337.5^2$. Taken together, these choices are meant to keep the loss terms on comparable scales so that one component does not overwhelm the others too early in training.

Interior collocation points are generated with Latin Hypercube Sampling over the full space-time domain. For the baseline hard/soft comparison, the model uses $N_f = 40{,}000$ uniform collocation points. In the resolution study, the sampling density is varied according to $N_f \in \{10{,}000, 20{,}000, 30{,}000, 40{,}000\}$, matching the four FEM resolution levels through the relation $N_f = 200\,N_x$. The network architecture remains fixed throughout that sweep so that the study isolates the effect of sampling density rather than mixing it with a change in model capacity. Figure 2 shows the training-point layout for the baseline case and makes clear how the initial data, boundary points, and interior collocation samples are distributed across the domain.

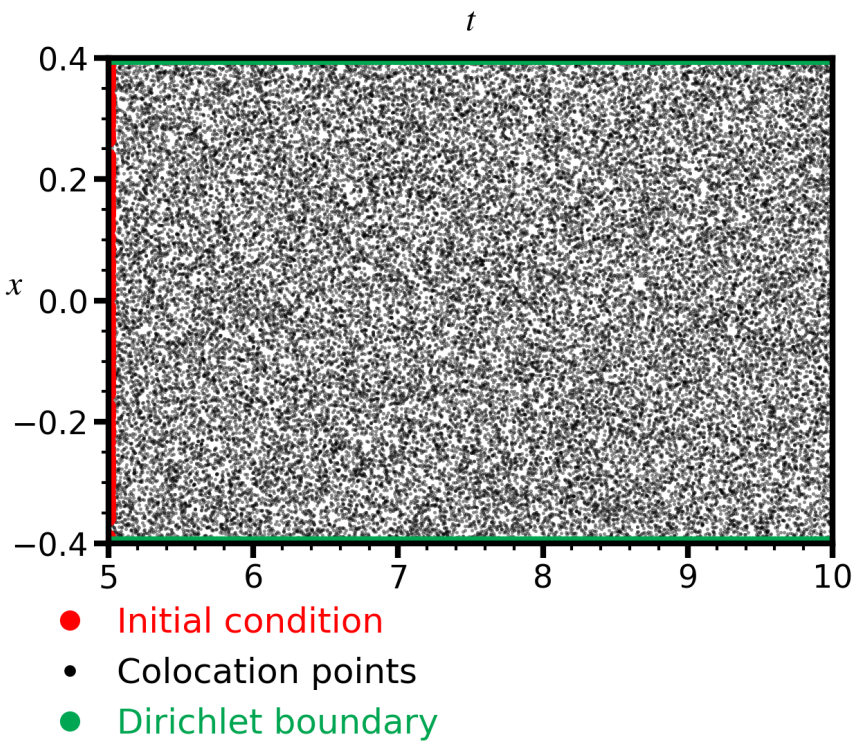


Figure 2: Training-point distribution in the space-time domain. Red points on the line $t = t_0$ represent the $N_0 = 150$ initial-condition samples, black points filling the interior are Latin-Hypercube collocation samples, and green points along the spatial boundaries are Dirichlet enforcement points used only for the soft-BC formulation.

3.2 FEM Reference Methodology

The FEM component serves two roles. First, it supplies the fine-grid temperature field in `thermal_fine.mat` (see Appendix A for its contents and provenance), which is used to initialize and validate the PINN. Second, it provides the coarse-grid reference solutions used in the refinement study. The FEM solver solves the same 1D phase-change problem on a nodal grid of N_x points over the interval $[-0.4, 0.4]$.

At the beginning of each FEM run, the initial temperature profile is obtained by cubic interpolation of the fine reference field onto the current coarse grid. Temperature-dependent properties are then updated from the current iterate. The thermal conductivity is treated piecewise: graphite uses the constant value 100 W/m K, while aluminum transitions between the solid and liquid conductivities using the same liquid-fraction rule as the PINN. Latent heat is handled through an effective heat-capacity term,

$$\rho c_{p,\mathrm{eff}} = \rho\left(c_p + L\frac{df_L}{dT}\right), \tag{9}$$

where df_L/dT is nonzero only inside the mushy zone. This is the FEM analogue of the enthalpy treatment used in the PINN residual and ensures that both models represent phase change in a consistent way. That consistency matters, because the FEM solution is used not only as a reference for accuracy but also as the source of the initial thermal profile fed into the PINN.

Time marching is performed implicitly. For each time step, midpoint conductivities and nodal heat-capacity contributions are assembled into a tridiagonal banded linear system. Because the material coefficients depend on the unknown temperature, the solver performs a

fixed-point iteration within each time step: starting from the previous solution, it repeatedly updates the coefficients, solves the banded system with `scipy.linalg.solve_banded`, and stops when the maximum nodal change falls below 10^{-6} or after ten iterations. The Dirichlet boundary conditions are imposed strongly by replacing the first and last rows of the system with the prescribed cold and hot temperatures. The result is a stable reference solution that can be refined in a controlled way and compared directly with the PINN predictions.

The FEM refinement study is carried out for $N_x \in \{50, 100, 150, 200\}$. In this study, the time grid is scaled with the spatial grid so that $N_t = N_x$, which keeps the refinement study balanced in space and time. To compare the coarse FEM solutions with the fine reference field and with the PINN predictions, each coarse FEM result is interpolated back onto the fine reference grid, first in space and then in time, before the relative L_2 error is computed. The same coarse solutions are also used to extract the liquidus isotherm and evaluate interface-position convergence.

4 Results

4.1 Loss Convergence

Figure 3 summarizes how the two boundary-condition strategies behave during training. The hard-BC model drives the pre-training loss down to approximately 10^{-8}, indicating that the initial condition is reproduced to near-machine precision before the PDE residual is activated. The soft-BC model also converges during pre-training, but it levels off closer to 10^{-7} because the boundary penalties introduce additional competing gradients even in this early stage.

Once the physics residual is switched on, both models exhibit a brief rise in loss followed by continued descent. The difference is in how they get there: the hard-BC curve remains comparatively smooth, whereas the soft-BC curve shows larger oscillations during the Adam phase. That behavior is consistent with the structure of the optimization problem, since the soft-BC network must simultaneously reconcile the PDE residual, the initial condition, and two boundary penalties. In other words, the soft-BC model is solving a more crowded optimization problem, and that extra competition shows up clearly in the training history.

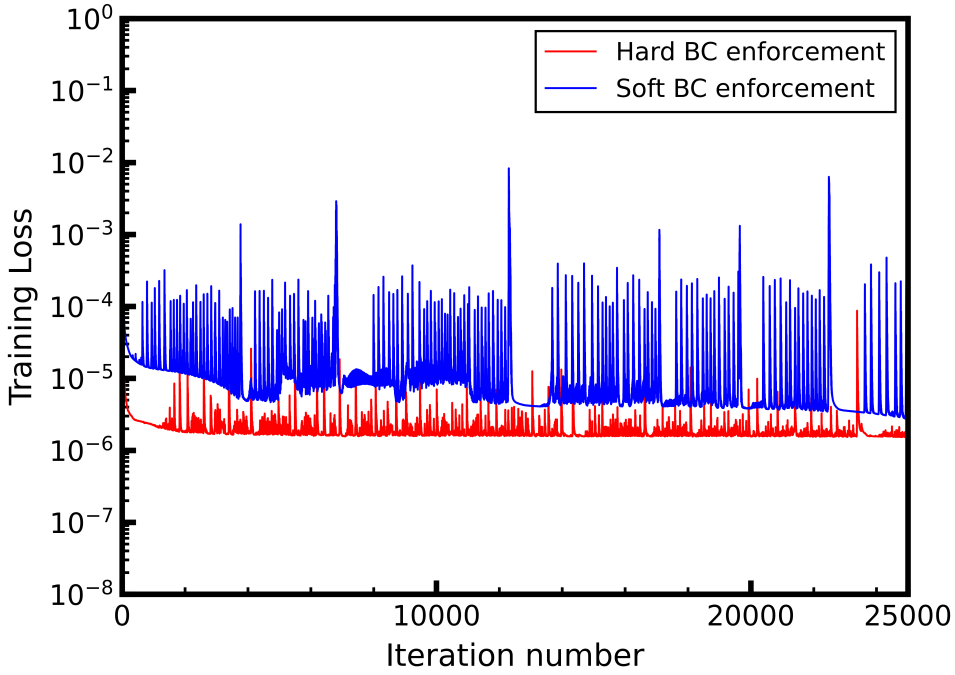


Figure 3: Training-loss histories for the hard-BC and soft-BC models. The hard-BC formulation reaches pre-training residuals near 10^{-8}, while the soft-BC formulation remains closer to 10^{-7} and displays larger Adam-phase oscillations because it optimizes a more competitive multi-term objective.

4.2 Temperature Field Accuracy

Figure 4 presents the temperature field predicted by the hard-BC model over the full space-time domain. The dominant thermal structure is captured clearly: the solution remains smooth through most of the rod, while a sharp gradient forms near the phase-change region where latent heat affects the local energy balance. Because the hard-BC formulation embeds the endpoint temperatures analytically, the predicted field also matches the prescribed boundary values exactly at all times.

In quantitative terms, the hard-BC PINN achieves a relative L_2 error of 0.9×10^{-3} against the FEM reference field at $N_f = 40{,}000$. This error level is small enough to reproduce the global temperature structure well, even though the most challenging part of the problem remains the narrow mushy zone around the moving interface. That pattern is consistent with the physics of the problem itself: most of the domain is comparatively smooth, while the phase-change region concentrates the strongest gradients.

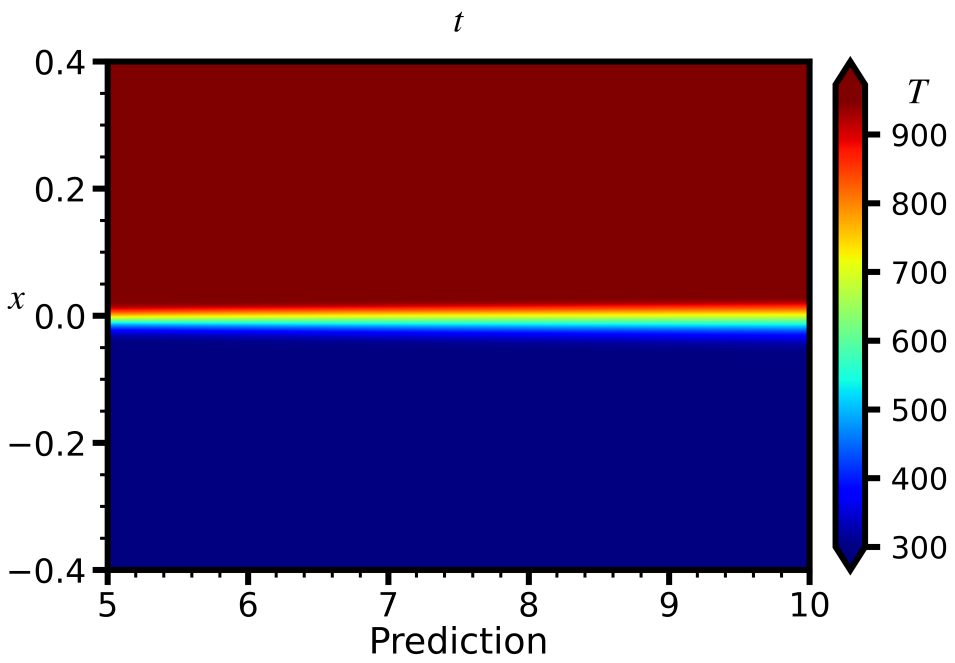


Figure 4: Temperature field predicted by the hard-BC PINN over $x \in [-0.4, 0.4]$ m and $t \in [5, 10]$ s. The smooth transition from the cold graphite side to the hot aluminum side is interrupted by a sharp gradient near $x = 0$, which marks the phase-change region where latent heat is active.

Figure 5 compares representative temperature profiles from the two PINN variants with the analytic Stefan solution. Both models reproduce the overall profile well across most of the domain, which reinforces the space-time picture shown in Figure 4. The main differences appear near the phase-transition region, where the mushy-zone approximation introduces a smooth thermal transition rather than the sharper behavior associated with an idealized interface description.

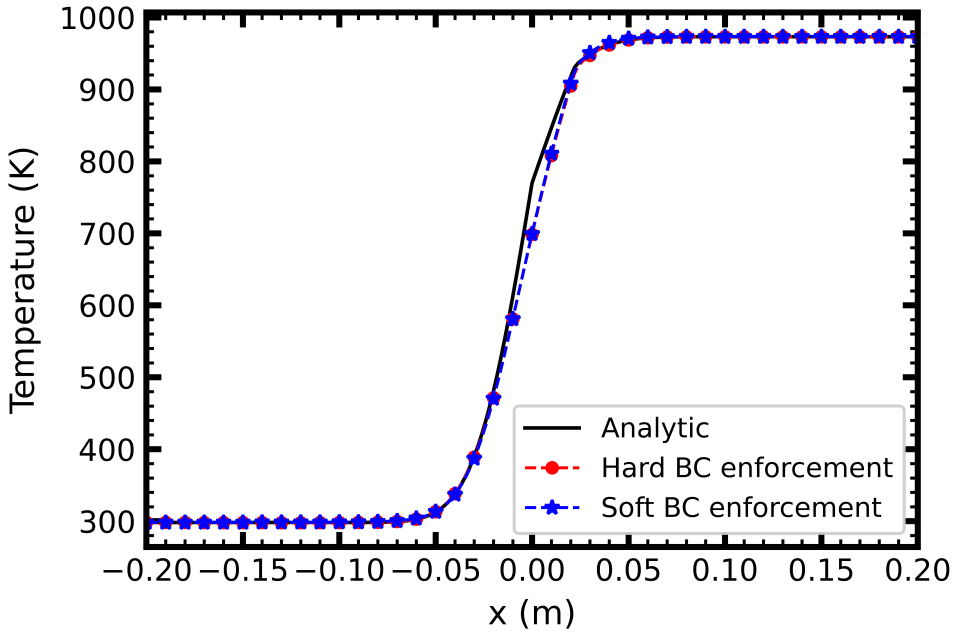


Figure 5: Representative temperature profiles for the hard-BC model (blue dashed), the soft-BC model (orange dashed), and the analytic Stefan solution (black solid). Both PINN variants capture the steep thermal gradient across the mushy zone, although the hard-BC profile remains slightly closer to the analytic curve near the domain boundaries.

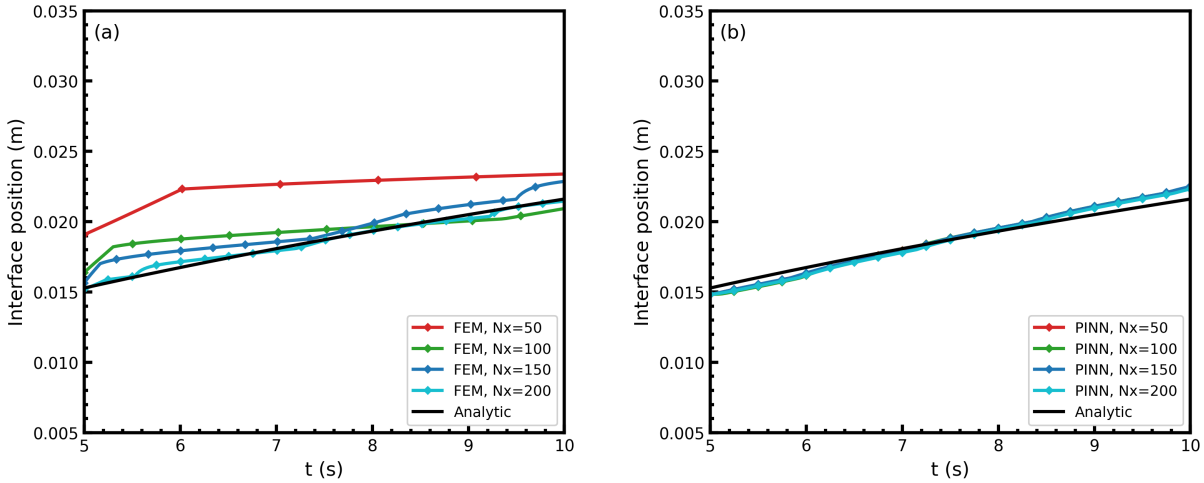
4.3 Phase-Interface Tracking

For this problem, the exact Stefan interface position is

$$s^*(t) = 7.095 \times 10^{-3} \sqrt{t}\ \text{m}, \tag{10}$$

derived from the Stefan condition for the specified material properties. In the numerical solutions, the interface is estimated by tracking the $T_l = 933.15\,\text{K}$ isotherm. This provides a consistent way to compare the PINN and FEM solutions with the analytic reference even though the benchmark uses a finite mushy-zone width rather than a perfectly sharp interface.

Figure 6 compares the PINN and FEM interface locations against $s^*(t)$ across all resolution levels.



(a) FEM — resolution convergence

(b) PINN — collocation density

Figure 6: Stefan interface position $s(t)$ identified through the $T_l = 933.15$ K isotherm. Panel (a) shows FEM results for $N_x \in \{50, 100, 150, 200\}$, where coarse meshes overshoot the analytic solution and then converge toward it with refinement. Panel (b) shows PINN results for $N_f \in \{10{,}000, \ldots, 40{,}000\}$; all four collocation levels track the analytic curve closely, illustrating the weak sensitivity of the PINN interface prediction to additional collocation points.

Both the PINN and FEM depart slightly from the sharp-interface analytic curve because the benchmark replaces the discontinuous phase boundary with a smooth 20 K mushy zone. Even with that modeling difference, the PINN interface remains close to the FEM result: at $N_f = 40{,}000$, the discrepancy stays within approximately 1 mm over the full interval $t \in [5, 10]$ s. This is a useful result, because it shows that the PINN is not only fitting temperatures pointwise but is also recovering a physically meaningful interface trajectory.

4.4 Collocation Resolution Study

Table 2 and Figure 7 summarize how the relative L_2 error changes with resolution in the FEM and PINN models.

Table 2: Relative L_2 temperature error for FEM and PINN (hard-BC and soft-BC) at matched resolution levels, with comparison to the benchmark values reported by Zhu et al. [9].

N_x	FEM L_2 (Ours)	FEM L_2 (Zhu et al [9])	PINN L_2 (Hard)	PINN L_2 (Soft)	Δ FEM vs. PINN
50	3.6×10^{-2}	3.7×10^{-2}	1.1×10^{-3}	2.4×10^{-3}	FEM $\gg$ PINN
100	9.1×10^{-3}	9.3×10^{-3}	1.0×10^{-3}	2.2×10^{-3}	FEM $>$ PINN
150	4.1×10^{-3}	4.2×10^{-3}	1.0×10^{-3}	2.1×10^{-3}	FEM $>$ PINN
200	2.3×10^{-3}	2.4×10^{-3}	0.9×10^{-3}	2.0×10^{-3}	FEM $\approx$ PINN

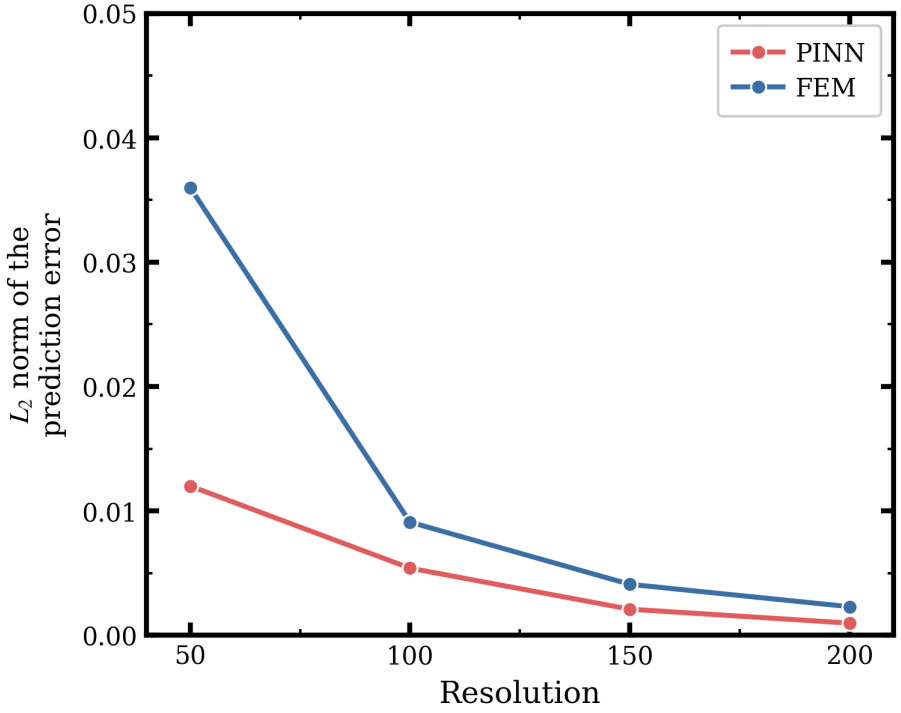


Figure 7: Relative L_2 error versus spatial resolution N_x for FEM (ours and paper benchmark), hard-BC PINN, and soft-BC PINN. The FEM curve decreases approximately as N_x^{-1}, consistent with first-order implicit Euler time-stepping when $N_t = N_x$, and closely tracks the values reported by Zhu et al. [9]. Both PINN variants remain nearly flat across the same sweep: the hard-BC model stays near 1×10^{-3} while the soft-BC model stays near 2×10^{-3}, showing that increasing the collocation count beyond $N_f = 10{,}000$ produces only marginal improvement. At coarse resolution the hard-BC PINN outperforms FEM by over an order of magnitude; at $N_x = 200$ the FEM error closes to within roughly $2.5\times$ of the hard-BC PINN.

The FEM results follow the expected convergence trend and agree closely with the values reported by Zhu et al. [9] (Table 2). Because the time-step count is coupled to the spatial resolution through $N_t = N_x$, the overall error scales approximately as N_x^{-1}, consistent with a first-order implicit Euler discretization. For example, increasing the mesh from $N_x = 50$ to $N_x = 100$ reduces the relative error by roughly $4\times$, from 3.6×10^{-2} to 9.1×10^{-3}, and further refinement continues that downward trend until the error reaches 2.3×10^{-3} at $N_x = 200$.

The PINN behaves differently. The hard-BC variant achieves a relative L_2 error of 1.1×10^{-3} at $N_f = 10{,}000$ collocation points, and increasing the count to 40,000 changes the result only marginally, to 0.9×10^{-3}. The soft-BC variant follows a similar flat trend, ranging from 2.4×10^{-3} to 2.0×10^{-3} across the same sweep. In both cases, once the network has seen a moderate number of collocation points, simply adding more samples does not produce classical mesh-like convergence. The model settles into an approximate solution quality that is largely insensitive to further sampling density, which is consistent with the observations reported by Zhu et al. [9].

This contrast is practically important. At the coarsest level ($N_x = 50$), the hard-BC PINN outperforms FEM by more than an order of magnitude (1.1×10^{-3} versus 3.6×10^{-2}), yet it does so without building a mesh. As resolution increases, FEM converges rapidly and closes the gap: at $N_x = 200$ the FEM error of 2.3×10^{-3} is comparable to the soft-BC PINN (2.0×10^{-3}) and only about $2.5\times$ larger than the hard-BC PINN (0.9×10^{-3}). For a 1D benchmark the computational implications are modest, but the balance between mesh-construction cost and

attainable accuracy becomes much more consequential in higher-dimensional AM simulations.

4.5 Implementation Findings

Two implementation issues proved especially important and are worth recording because they affected the interpretation of the results as much as the raw training curves did.

Soft-BC saddle collapse. In early experiments, the soft-BC model was pre-trained with interface-refined collocation, which placed extra points near the expected phase boundary before the network had learned the global thermal field. Under those conditions, the model repeatedly collapsed to a nearly uniform temperature distribution around $T \approx 666$ K, close to the spatial average of the two boundary values. That solution was clearly unphysical, yet it could still produce a deceptively small normalized residual because a nearly constant field weakens the temporal derivative terms in the PDE.

The collapse was eliminated only when three modifications were applied together: first, pre-training was restricted to uniformly distributed collocation points with no interface bias; second, the output scaling of Eq. (5) was added so that the network prediction started in a physically meaningful temperature range; third, the PDE residual was normalized by T_{NRM}. Using only one or two of these changes was not sufficient, which highlights how sensitive soft boundary enforcement can be to the balance of the optimization problem.

FEM reference quality. The second issue involved the quality of the FEM reference itself. In the original code, the time-step count was fixed at $N_t = 201$ for every spatial resolution N_x. At coarse meshes, that choice made the temporal discretization artificially fine relative to the spatial discretization, which blurred the interpretation of the convergence study. Setting $N_t = N_x$ instead coarsened space and time together, producing a more consistent resolution sweep and aligning the FEM trend with the benchmark behavior reported in Figure 6 of Zhu et al. [9].

5 Conclusion

This study shows that hard boundary enforcement is the more reliable choice for the 1D AM Stefan benchmark considered here. Reimplementing the Zhu et al. [9] problem in PyTorch made it possible to compare hard and soft boundary strategies under the same network architecture, sampling design, and training protocol. The resulting picture is consistent across the loss histories, the field predictions, and the debugging process itself: embedding the boundary values directly into the model removes a major source of optimization conflict and leads to more stable training.

The accuracy results reinforce that conclusion. The hard-BC model pre-trains more cleanly, reaches residuals near 10^{-8}, and avoids the collapse behavior that can affect the soft-BC formulation. At low resolution, the two PINN variants deliver similar accuracy, but the hard-BC formulation does so with less tuning and less sensitivity to initialization. For practical use, that difference in reliability matters as much as the final error value.

The collocation study further shows that this benchmark is not limited by point count alone. Increasing N_f from 10,000 to 40,000 produces almost no systematic improvement in PINN accuracy (hard-BC: 1.1×10^{-3} to 0.9×10^{-3}; soft-BC: 2.4×10^{-3} to 2.0×10^{-3}), whereas FEM continues to improve with refinement from 3.6×10^{-2} to 2.3×10^{-3}. At coarse resolution the

PINN outperforms FEM by over an order of magnitude while avoiding explicit mesh construction, which is precisely the tradeoff that makes physics-informed models attractive for more complex AM geometries.

At the same time, the benchmark remains deliberately simple. It is one dimensional, it uses fixed boundary temperatures rather than a moving heat source, and it initializes the thermal field from an FEM solution instead of experimental data. Those simplifications make the study valuable as a controlled comparison, but they also limit how directly the conclusions can be transferred to real manufacturing settings. The soft-BC saddle collapse is an especially clear reminder that training sensitivity is likely to increase, not decrease, as the physics become more realistic.

Future work should therefore focus on strategies that improve accuracy where the present model struggles most: near the phase boundary. Adaptive collocation, as explored by Madir et al. [3], is a natural next step because it targets the stiffest region of the domain without requiring a proportional increase in total samples. Extending the approach to multi-layer thermal histories of the kind studied by Peng and Panesar [4] would test whether the hard-BC advantage persists in more realistic AM scenarios. Beyond that, uncertainty quantification, highlighted as a major gap by Farrag et al. [2], will be essential if PINN-based thermal prediction is to become a decision tool rather than only a numerical demonstration.

Author Contribution Statement

Goutam Kumar Biswas — PINN implementation in PyTorch (full source listed in Appendix B), including the hard-BC and soft-BC variants, training pipeline, collocation-resolution study, and figure-generation workflow; formal analysis of PINN behavior and results; writing of the original draft and subsequent revisions.

Maddox Gonzalez — FEM implementation and validation, including reference-solution generation, mesh-refinement analysis, and interpretation of FEM convergence behavior; review and editing of the manuscript.

Acknowledgements

This work was completed as the graduate project for EML 5360 at the University of Central Florida in Spring 2026. The authors sincerely thank Dr. Dazhong Wu for his instruction, guidance, and the course environment that motivated this project. The authors also gratefully acknowledge the work of Qiming Zhu et al. [9], whose formulation of PINN-based thermal modeling for additive manufacturing provided the foundation for this reimplementation. Finally, the authors thank the open-source communities behind PyTorch, NumPy, SciPy, and Matplotlib, whose tools made the implementation and analysis substantially more efficient.

References

[1] Shengze Cai, Zhicheng Wang, Sifan Wang, Paris Perdikaris, and George Em Karniadakis. Physics-informed neural networks for heat transfer problems. *Journal of Heat Transfer*, 143(6):060801, 2021. doi: 10.1115/1.4050542.

[2] Abdelrahman Farrag, Yuxin Yang, Nieqing Cao, Daehan Won, and Yu Jin. Physics-informed machine learning for metal additive manufacturing. *Progress in Additive Manufacturing*, 10(1):171–185, 2025. doi: 10.1007/s40964-024-00612-1.

[3] Bahae-Eddine Madir, Francky Luddens, Corentin Lothodé, and Ionut Danaila. Physics informed neural networks for heat conduction with phase change. *International Journal of Heat and Mass Transfer*, 252:127430, 2025. doi: 10.1016/j.ijheatmasstransfer.2025.127430.

[4] Bohan Peng and Ajit Panesar. Multi-layer thermal simulation using physics-informed neural network. *Additive Manufacturing*, 95:104498, 2024. doi: 10.1016/j.addma.2024.104498.

[5] Maziar Raissi, Paris Perdikaris, and George E. Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019. doi: 10.1016/j.jcp.2018.10.045.

[6] V. R. Voller, M. Cross, and N. C. Markatos. An enthalpy method for convection/diffusion phase change. *International Journal for Numerical Methods in Engineering*, 24:271–284, 1987. doi: 10.1002/nme.1620240119.

[7] Sifan Wang and Paris Perdikaris. Deep learning of free boundary and Stefan problems. *Journal of Computational Physics*, 428:109914, 2021. doi: 10.1016/j.jcp.2020.109914.

[8] Sifan Wang, Yujun Teng, and Paris Perdikaris. Understanding and mitigating gradient flow pathologies in physics-informed neural networks. *SIAM Journal on Scientific Computing*, 43(5):A3055–A3081, 2021. doi: 10.1137/20M1318043.

[9] Qiming Zhu, Zeliang Liu, and Jinhui Yan. Machine learning for metal additive manufacturing: predicting temperature and melt pool fluid dynamics using physics-informed neural networks. *Computational Mechanics*, 67(2):619–635, 2021. doi: 10.1007/s00466-020-01952-9.

A FEM Pre-Training Data

Before the PINN sees a single physics residual, it spends a short warm-up phase learning to mimic a finite element solution of the same 1D Stefan problem. That reference solution lives in a single MATLAB file, `thermal_fine.mat`, which was not generated as part of this work: it comes straight from the public dataset that Zhu et al. released with the original PINN-AM paper [9], hosted at `https://github.com/QimingZhu1992/PINN-AM/tree/main/1D/dat`. The file is bundled with this submission so the notebook in Appendix B can be re-run without any external download.

Inside the file are three arrays, listed in Table 3. The spatial coordinate `x` runs from -0.4 m to 0.4 m on a uniform 401-point grid, the time vector `tt` spans 5 to 10 s on a uniform 201-point grid, and the temperature field `Tem` is the corresponding 401×201 FEM solution, with values bounded between the ambient 298.15 K and the imposed hot-side 973.15 K. Together those arrays give roughly eighty thousand (x, t, T) samples covering the full space-time window of interest.

Table 3: Variables stored in `thermal_fine.mat`.

Variable	Shape	Range	Description
`x`	1×401	$[-0.4, 0.4]$ m	Spatial grid (uniform, $\Delta x = 2 \times 10^{-3}$ m)
`tt`	1×201	$[5, 10]$ s	Time grid (uniform, $\Delta t = 2.5 \times 10^{-2}$ s)
`Tem`	401×201	$[298.15, 973.15]$ K	FEM temperature field $T(x_i, t_j)$

In practice the dataset is consumed in two ways. During pre-training, the file is loaded with `scipy.io.loadmat`, the `x` and `tt` vectors are flattened into column form, paired up over the full grid, and fed to the network with the corresponding `Tem` values as a plain supervised regression problem. This gives the PINN a sensible starting point—the rough shape of the temperature field is already in place—before the PDE residual loss is switched on. After training, the same grid is reused as the ground truth against which the relative L_2 error reported in Section 4 is computed, so pre-training data and evaluation data come from exactly the same source. The exact loading routine and tensor construction can be found in Appendix B.

B PINN Implementation Source Code

The complete PyTorch implementation used to produce the results in this paper is reproduced below. The source is taken verbatim from the Jupyter notebook `PINN_AM_PyTorch.ipynb` accompanying this submission, with code cells labeled in the order in which they appear in the notebook.

```
# ===== Cell 1 =====
# -- 0. Google Colab: Mount Drive ---------------------------------------------
try:
    from google.colab import drive
    drive.mount('/content/drive')
    import os
    os.chdir('/content/drive/MyDrive/EML 5360/PINN-AM-main')
    print('Working directory:', os.getcwd())
except ModuleNotFoundError:
    print('Not on Colab - skipping Drive mount.')

# ===== Cell 2 =====
# -- 1a. Install / Upgrade Dependencies ----------------------------------------
# NumPy and SciPy must be installed together to avoid ABI mismatch
# (e.g. "cannot import name '_center' from 'numpy._core.umath'").
import subprocess, sys, importlib

def _pip_install(args):
    r = subprocess.run(
        [sys.executable, '-m', 'pip', 'install', *args, '-q'],
        capture_output=True, text=True,
    )
    return 'Successfully installed' in (r.stdout + r.stderr)

_NEED_RESTART = False

# 1) Check NumPy/SciPy ABI compatibility; if broken, force-reinstall both together.
_abi_ok = True
try:
```

```
    importlib.import_module('numpy')
    importlib.import_module('scipy.io')
except Exception as e:
    print(f'  ? NumPy/SciPy import failed ({type(e).__name__}): {e}')
    _abi_ok = False

if not _abi_ok:
    print('  ? Force-reinstalling numpy + scipy together to fix ABI mismatch...')
    if _pip_install(['--force-reinstall', '--no-cache-dir',
                     'numpy>=2.0', 'scipy>=1.14.0']):
        _NEED_RESTART = True
        print('  ? reinstalled: numpy + scipy')

# 2) Upgrade remaining packages individually.
for pkg in ['matplotlib', 'pyDOE']:
    if _pip_install(['--upgrade', pkg]):
        _NEED_RESTART = True
        print(f'  ? upgraded: {pkg}')

if _NEED_RESTART:
    print('\n??  Packages were installed/upgraded - restarting runtime now...')
    print('   After restart, re-run from this cell (it will skip the restart).')
    import os; os.kill(os.getpid(), 9)     # Colab-safe hard restart
else:
    print('? All packages up to date - no restart needed.')

# ===== Cell 3 =====
# -- 1b. Imports ------------------------------------------------------------
import torch
import torch.nn as nn
import numpy as np
import scipy.io
import matplotlib
import matplotlib.pyplot as plt
from matplotlib.ticker import AutoMinorLocator
from mpl_toolkits.axes_grid1 import make_axes_locatable
from scipy.interpolate import RegularGridInterpolator, interp1d
from scipy.interpolate import interp1d as _ic_interp
from scipy.linalg      import solve_banded as _solve_banded
from pyDOE import lhs
import os, time, warnings, logging
warnings.filterwarnings('ignore')
logging.getLogger('matplotlib.font_manager').setLevel(logging.ERROR)

print(f'PyTorch   : {torch.__version__}')
print(f'NumPy     : {np.__version__}')
print(f'Matplotlib: {matplotlib.__version__}')

# -- 2. Reproducibility, Device, and Plot Style -----------------------------
SEED = 42
torch.manual_seed(SEED); np.random.seed(SEED)

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
if device.type == 'cuda':
    torch.set_float32_matmul_precision('high')
    torch.backends.cuda.matmul.allow_tf32 = True
    torch.backends.cudnn.allow_tf32       = True
    torch.backends.cudnn.benchmark        = True
```

```
    torch.backends.cudnn.deterministic    = False
    gpu  = torch.cuda.get_device_properties(0)
    print(f'Running on : {gpu.name}  ({gpu.total_memory/1024**3:.1f} GB VRAM)')
else:
    torch.backends.cudnn.deterministic = True
    torch.backends.cudnn.benchmark     = False
    print('Running on : CPU  (no CUDA device found)')

plt.rcParams.update({
    'font.family': 'serif', 'font.serif': ['Times New Roman', 'DejaVu Serif', 'serif'],
    'mathtext.fontset': 'stix', 'font.size': 11, 'axes.labelsize': 12,
    'axes.titlesize': 12, 'xtick.labelsize': 10, 'ytick.labelsize': 10,
    'legend.fontsize': 10, 'legend.framealpha': 0.9,
    'lines.linewidth': 2.0, 'axes.linewidth': 2.0,
    'xtick.major.width': 2.0, 'ytick.major.width': 2.0,
    'xtick.minor.width': 1.0, 'ytick.minor.width': 1.0,
    'xtick.major.size': 5.0, 'ytick.major.size': 5.0,
    'xtick.minor.size': 2.5, 'ytick.minor.size': 2.5,
    'figure.dpi': 600, 'savefig.dpi': 1200, 'savefig.bbox': 'tight', 'savefig.pad_inches':
     0.05,
})


# ===== Cell 4 =====

# -- 3. Load FEM Reference Data ------------------------------------------------
DATA_PATH = '1D/dat/thermal_fine.mat'
data  = scipy.io.loadmat(DATA_PATH)
x_fem = data['x'].flatten()[:, None]
t_fem = data['tt'].flatten()[:, None]
T_fem = data['Tem']
xx    = x_fem.flatten()
tt    = t_fem.flatten()
Nx, Nt = len(xx), len(tt)
X_mesh, T_mesh = np.meshgrid(xx, tt)
X_star  = np.hstack((X_mesh.flatten()[:,None], T_mesh.flatten()[:,None]))
T_exact = T_fem.T   # (Nt, Nx)
print(f'FEM grid  : {Nx} x-nodes x {Nt} time-steps')
print(f'x range   : [{xx.min():.3f}, {xx.max():.3f}] m')
print(f't range   : [{tt.min():.2f}, {tt.max():.2f}] s')

# -- 4. Physics-Informed Neural Network (Hard BC) ------------------------------
class PhysicsInformedNN(nn.Module):
    RHO_AL, RHO_GRAP            = 2555.0,  2200.0
    KAPPA_AL_LIQ, KAPPA_AL_SOL  = 91.0,    211.0
    KAPPA_GRAP                  = 100.0
    CP_AL, CP_GRAP              = 1190.0,  1700.0
    CL_AL                       = 3.98e5
    TS, TL                      = 913.15,  933.15
    T_COLD, T_HOT               = 298.15,  973.15
    XLEN                        = 0.8

    def __init__(self, layers, lb, ub):
        super().__init__()
        self.lb = torch.tensor(lb, dtype=torch.float32, device=device)
        self.ub = torch.tensor(ub, dtype=torch.float32, device=device)
        self.linears = nn.ModuleList(
            [nn.Linear(layers[i], layers[i+1]) for i in range(len(layers)-1)])
```

```
        for layer in self.linears[:-1]:
            nn.init.xavier_normal_(layer.weight, gain=1.0)
            nn.init.zeros_(layer.bias)
        nn.init.xavier_normal_(self.linears[-1].weight)
        nn.init.zeros_(self.linears[-1].bias)

    def _raw_net(self, x, t):
        X = torch.cat([x, t], dim=1)
        H = 2.0 * (X - self.lb) / (self.ub - self.lb) - 1.0
        for layer in self.linears[:-1]:
            H = torch.tanh(layer(H))
        return self.linears[-1](H)

    def _smooth_H(self, x):
        eps  = 1e-3
        d1   = x - (-0.4 + eps)
        d2   = (0.4 - eps) - x
        dist = torch.minimum(d1, d2)
        Hcal = 0.5 * (1.0 + dist / eps + (1.0 / torch.pi) * torch.sin(dist * torch.pi /
    eps))
        return torch.where(dist < -eps, torch.zeros_like(x),
               torch.where(dist >  eps, torch.ones_like(x), Hcal))

    def net_T(self, x, t):
        H_mask = self._smooth_H(x)
        Tbc    = self.T_COLD + (self.T_HOT - self.T_COLD) / self.XLEN * (x + 0.4)
        return Tbc * (1.0 - H_mask) + self._raw_net(x, t) * H_mask

    def net_f(self, x_in, t_in):
        x = x_in.clone().detach().requires_grad_(True)
        t = t_in.clone().detach().requires_grad_(True)
        T  = self.net_T(x, t)
        fL = torch.clamp((T - self.TS) / (self.TL - self.TS), 0.0, 1.0)
        def g(out, inp):
            return torch.autograd.grad(out.sum(), inp, create_graph=True)[0]
        T_t  = g(T,  t);  fL_t = g(fL, t);  T_x = g(T, x)
        al   = (x > 0.0).float();  al_c = 1.0 - al
        rho   = self.RHO_AL   * al + self.RHO_GRAP * al_c
        kappa = (self.KAPPA_AL_LIQ * fL + self.KAPPA_AL_SOL * (1.0 - fL)) * al \
              + self.KAPPA_GRAP * al_c
        cp    = self.CP_AL    * al + self.CP_GRAP  * al_c
        lap   = g(kappa * T_x, x)
        return (rho * cp * T_t + rho * self.CL_AL * fL_t - lap) / \
               (self.RHO_AL * self.KAPPA_AL_SOL)

    @torch.no_grad()
    def predict_T(self, X_np):
        x = torch.tensor(X_np[:, 0:1], dtype=torch.float32, device=device)
        t = torch.tensor(X_np[:, 1:2], dtype=torch.float32, device=device)
        return self.net_T(x, t).cpu().numpy()

    def predict_f(self, X_np):
        x = torch.tensor(X_np[:, 0:1], dtype=torch.float32, device=device)
        t = torch.tensor(X_np[:, 1:2], dtype=torch.float32, device=device)
        with torch.enable_grad():
            f = self.net_f(x, t)
        return f.detach().cpu().numpy()

```

```python
# -- 5. Training Data Sampling and Trainer -----------------------------------
from scipy.optimize import minimize as scipy_minimize

def sample_training_data(N0=300, N_f=40_000,
                         lb=np.array([-0.4, 5.0]), ub=np.array([0.4, 10.0])):
    idx  = np.random.choice(Nx, N0, replace=False)
    x0   = x_fem[idx, :]
    T0   = T_fem[idx, 0:1]
    Xf   = lb + (ub - lb) * lhs(2, N_f)
    Xf   = Xf[np.argsort(Xf[:, 0])]
    return x0, T0, Xf[:, 0:1], Xf[:, 1:2]

class Trainer:
    def __init__(self, model, x0, T0, xf, tf, lambda_f=1e-3, X_tem=None):
        self.model    = model.to(device)
        self.lambda_f = lambda_f
        def _t(a): return torch.tensor(a, dtype=torch.float32, device=device)
        self.x0   = _t(x0);  self.t0 = torch.full_like(self.x0, 5.0)
        self.T0   = _t(T0);  self.xf = _t(xf);  self.tf = _t(tf)
        self.X_tem = _t(X_tem) if X_tem is not None else None
        self.loss_log = []

    def loss_pre(self):
        T_pred = self.model.net_T(self.xf, self.tf)
        return torch.mean((T_pred - self.X_tem) ** 2) / T_NRM

    def loss(self):
        T0_pred = self.model.net_T(self.x0, self.t0)
        L_ic = torch.mean((T0_pred - self.T0) ** 2) / T_NRM
        f    = self.model.net_f(self.xf, self.tf)
        L_f  = self.lambda_f * torch.mean(f ** 2)
        return L_ic + L_f

    def train_adam(self, n_iter=10_000, lr=1e-3, log_every=1000, loss_fn=None, clip=None):
        if loss_fn is None: loss_fn = self.loss
        opt = torch.optim.Adam(self.model.parameters(), lr=lr)
        t0  = time.time()
        for it in range(1, n_iter + 1):
            opt.zero_grad(); L = loss_fn(); L.backward()
            if clip is not None:
                torch.nn.utils.clip_grad_norm_(self.model.parameters(), clip)
            opt.step(); self.loss_log.append(L.item())
            if it % log_every == 0 or it == 1:
                print(f'  Adam {it:6d}/{n_iter} | loss {L.item():.4e} | {time.time()-t0:.1
    f}s')
        print(f'Adam done. Final loss: {self.loss_log[-1]:.4e}')

    def train_lbfgs(self, max_iter=50_000, log_every=1000, loss_fn=None):
        if loss_fn is None: loss_fn = self.loss
        k = [0]
        def _get_params():
            return np.concatenate([p.detach().cpu().numpy().flatten()
                                   for p in self.model.parameters()]).astype(np.float64)
        def _set_params(params):
            offset = 0
            for p in self.model.parameters():
                n = p.numel()
                p.data.copy_(torch.tensor(params[offset:offset+n],
```

```
                            dtype=torch.float32).reshape(p.shape).to(device))
                offset += n
        def loss_and_grad(params):
            _set_params(params)
            self.model.zero_grad(); L = loss_fn(); L.backward()
            grad = np.concatenate([p.grad.detach().cpu().numpy().flatten()
                                   for p in self.model.parameters()])
            val = L.item(); self.loss_log.append(val); k[0] += 1
            if k[0] % log_every == 0:
                print(f'  L-BFGS-B {k[0]:6d} | loss {val:.4e}')
            return val, grad.astype(np.float64)
        result = scipy_minimize(loss_and_grad, _get_params(), method='L-BFGS-B', jac=True,
            options={'maxiter': max_iter, 'maxfun': max_iter, 'maxcor': 50,
                     'maxls': 50, 'ftol': np.finfo(float).eps})
        _set_params(result.x)
        print(f'L-BFGS-B done. {result.message}')
        print(f'Final loss: {result.fun:.4e}  |  iterations: {k[0]}')

# -- 6. Hyperparameters ------------------------------------------------------
FAST_MODE = False
if FAST_MODE:
    N0, N_f              = 200,  10_000
    N_PRE_ADAM, N_PRE_LBFGS = 1_000, 5_000
    N_ADAM, N_LBFGS      = 5_000, 5_000
else:
    N0, N_f              = 150,  40_000
    N_PRE_ADAM, N_PRE_LBFGS = 50_000, 50_000
    N_ADAM, N_LBFGS      = 50_000, 50_000

LAMBDA_F = 1e-8          # scaled by 1/T_NRM to maintain balance after normalization
T_NRM    = 337.5 ** 2    # (T_HOT - T_COLD)^2 / 4; normalizes all temperature MSE losses
LAYERS   = [2, 200, 200, 200, 200, 1]
lb       = np.array([-0.4,  5.0])
ub       = np.array([ 0.4, 10.0])

x0_np, T0_np, xf_np, tf_np = sample_training_data(N0=N0, N_f=N_f, lb=lb, ub=ub)
os.makedirs('cache', exist_ok=True); os.makedirs('figures', exist_ok=True)

# IC temperature at collocation x-positions (for pre-training)
_ic_fn   = interp1d(xx, T_exact[0], kind='cubic', fill_value='extrapolate')
X_tem_np = _ic_fn(xf_np[:, 0])[:, None].astype(np.float32)

print(f'Architecture : {LAYERS}')
print(f'N_f={N_f:,} | N0={N0} | Pre-Adam={N_PRE_ADAM:,} | Pre-L-BFGS={N_PRE_LBFGS:,}')
print(f'Main-Adam={N_ADAM:,} | Main-L-BFGS={N_LBFGS:,}')

# -- 7. Train Hard BC Model --------------------------------------------------
CACHE_HARD = 'cache/hard_bc.pt'
model   = PhysicsInformedNN(LAYERS, lb, ub).to(device)
trainer = Trainer(model, x0_np, T0_np, xf_np, tf_np, lambda_f=LAMBDA_F, X_tem=X_tem_np)

if os.path.exists(CACHE_HARD) and os.path.getsize(CACHE_HARD) > 0:
    ckpt = torch.load(CACHE_HARD, map_location=device, weights_only=False)
    model.load_state_dict(ckpt['model']); trainer.loss_log = ckpt['loss_log']
    trainer.pre_loss_log = ckpt.get('pre_loss_log', [])
    print(f'[OK] Loaded hard BC from cache ({len(trainer.pre_loss_log)}+{len(trainer.
    loss_log)} loss entries). Skipping training.')
else:
```

```
    print('Hard BC: no cache. Training...')
    print('--- Pre-train Stage 1: Adam ---')
    trainer.train_adam(n_iter=N_PRE_ADAM, lr=1e-3, log_every=500, loss_fn=trainer.loss_pre
    )
    print('--- Pre-train Stage 2: L-BFGS-B ---')
    trainer.train_lbfgs(max_iter=N_PRE_LBFGS, log_every=500, loss_fn=trainer.loss_pre)
    trainer.pre_loss_log = list(trainer.loss_log)
    trainer.loss_log = []
    print('--- Main Stage 1: Adam ---')
    trainer.train_adam(n_iter=N_ADAM, lr=1e-3, log_every=1000)
    print('--- Main Stage 2: L-BFGS-B ---')
    trainer.train_lbfgs(max_iter=N_LBFGS, log_every=500)
    torch.save({'model': model.state_dict(), 'loss_log': trainer.loss_log,
                'pre_loss_log': trainer.pre_loss_log}, CACHE_HARD)
    print(f'Hard BC model saved -> {CACHE_HARD}')

# -- 8. Soft BC Model and Trainer ------------------------------------------
class PhysicsInformedNN_Soft(nn.Module):
    RHO_AL, RHO_GRAP            = 2555.0,  2200.0
    KAPPA_AL_LIQ, KAPPA_AL_SOL  = 91.0,    211.0
    KAPPA_GRAP                  = 100.0
    CP_AL, CP_GRAP              = 1190.0,  1700.0
    CL_AL                       = 3.98e5
    TS, TL                      = 913.15,  933.15
    T_COLD, T_HOT               = 298.15,  973.15

    def __init__(self, layers, lb, ub):
        super().__init__()
        self.lb = torch.tensor(lb, dtype=torch.float32, device=device)
        self.ub = torch.tensor(ub, dtype=torch.float32, device=device)
        self.linears = nn.ModuleList(
            [nn.Linear(layers[i], layers[i+1]) for i in range(len(layers)-1)])
        for layer in self.linears[:-1]:
            nn.init.xavier_normal_(layer.weight, gain=1.0); nn.init.zeros_(layer.bias)
        nn.init.xavier_normal_(self.linears[-1].weight); nn.init.zeros_(self.linears[-1].
    bias)

    def net_T(self, x, t):
        # Scale output to physical temperature range so last-layer weights stay O(1)
        # and prevent tanh saturation saddle at initialization
        T_ref   = 0.5 * (self.T_HOT + self.T_COLD)   # 635.65 K
        T_scale = 0.5 * (self.T_HOT - self.T_COLD)   # 337.5 K
        X = torch.cat([x, t], dim=1)
        H = 2.0 * (X - self.lb) / (self.ub - self.lb) - 1.0
        for layer in self.linears[:-1]:
            H = torch.tanh(layer(H))
        return self.linears[-1](H) * T_scale + T_ref

    def net_f(self, x_in, t_in):
        x = x_in.clone().detach().requires_grad_(True)
        t = t_in.clone().detach().requires_grad_(True)
        T  = self.net_T(x, t)
        fL = torch.clamp((T - self.TS) / (self.TL - self.TS), 0.0, 1.0)
        def g(out, inp):
            return torch.autograd.grad(out.sum(), inp, create_graph=True)[0]
        T_t = g(T, t); fL_t = g(fL, t); T_x = g(T, x)
        al  = (x > 0.0).float(); al_c = 1.0 - al
        rho   = self.RHO_AL   * al + self.RHO_GRAP * al_c
```

```
        kappa = (self.KAPPA_AL_LIQ * fL + self.KAPPA_AL_SOL * (1.0 - fL)) * al \
                + self.KAPPA_GRAP * al_c
        cp    = self.CP_AL    * al + self.CP_GRAP  * al_c
        lap   = g(kappa * T_x, x)
        return (rho * cp * T_t + rho * self.CL_AL * fL_t - lap) / \
               (self.RHO_AL * self.KAPPA_AL_SOL)

    @torch.no_grad()
    def predict_T(self, X_np):
        x = torch.tensor(X_np[:, 0:1], dtype=torch.float32, device=device)
        t = torch.tensor(X_np[:, 1:2], dtype=torch.float32, device=device)
        return self.net_T(x, t).cpu().numpy()

class TrainerSoft(Trainer):
    def __init__(self, model, x0, T0, xf, tf, lambda_f=1e-3, lambda_bc=1.0,
                 lb=None, ub=None, X_tem=None):
        super().__init__(model, x0, T0, xf, tf, lambda_f, X_tem=X_tem)
        self.lambda_bc = lambda_bc
        # xf/tf are the full 160k points (uniform + interface-refined).
        # Pre-training must use ONLY the first n_pre uniform points to avoid the
        # zero-gradient saddle caused by the 120k concentrated interface points
        # pulling the network to output mean(T_IC) over all 160k targets.
        n_pre = len(X_tem) if X_tem is not None else len(xf)
        self.xf_pre = self.xf[:n_pre]
        self.tf_pre = self.tf[:n_pre]
        n_bc  = 150
        t_bc  = np.random.uniform(lb[1], ub[1], (n_bc, 1)).astype(np.float32)
        def _t(a): return torch.tensor(a, dtype=torch.float32, device=device)
        self.x_L  = _t(np.full((n_bc, 1), lb[0], dtype=np.float32))
        self.x_R  = _t(np.full((n_bc, 1), ub[0], dtype=np.float32))
        self.t_bc = _t(t_bc)
        self.T_cold = model.T_COLD; self.T_hot = model.T_HOT

    def loss_pre(self):
        # Pre-train on uniform points only (matches X_tem size = n_pre)
        T_pred = self.model.net_T(self.xf_pre, self.tf_pre)
        return torch.mean((T_pred - self.X_tem) ** 2) / T_NRM

    def loss(self):
        T0_pred = self.model.net_T(self.x0, self.t0)
        L_ic = torch.mean((T0_pred - self.T0) ** 2) / T_NRM
        L_f  = self.lambda_f  * torch.mean(self.model.net_f(self.xf, self.tf) ** 2)
        L_bc = self.lambda_bc * (
            torch.mean((self.model.net_T(self.x_L, self.t_bc) - self.T_cold) ** 2) +
            torch.mean((self.model.net_T(self.x_R, self.t_bc) - self.T_hot)  ** 2)) /
    T_NRM
        return L_ic + L_f + L_bc

lbr = np.array([-0.05, lb[1]]); ubr = np.array([0.05, ub[1]])
Xf_refined = lbr + (ubr - lbr) * lhs(2, 3 * N_f)
Xf_refined = Xf_refined[np.argsort(Xf_refined[:, 0])]
xf_soft_np = np.concatenate([xf_np, Xf_refined[:, 0:1]], axis=0)
tf_soft_np = np.concatenate([tf_np, Xf_refined[:, 1:2]], axis=0)

# IC temperature at soft collocation x-positions (for pre-training)
X_tem_soft_np = _ic_fn(xf_np[:, 0])[:, None].astype(np.float32)  # 40k uniform only

CACHE_SOFT = 'cache/soft_bc.pt'
```

```
torch.manual_seed(SEED)
model_soft   = PhysicsInformedNN_Soft(LAYERS, lb, ub).to(device)
trainer_soft = TrainerSoft(model_soft, x0_np, T0_np, xf_soft_np, tf_soft_np,
                           lambda_f=LAMBDA_F, lambda_bc=1.0, lb=lb, ub=ub,
                           X_tem=X_tem_soft_np)

if os.path.exists(CACHE_SOFT) and os.path.getsize(CACHE_SOFT) > 0:
    ckpt = torch.load(CACHE_SOFT, map_location=device, weights_only=False)
    model_soft.load_state_dict(ckpt['model']); trainer_soft.loss_log = ckpt['loss_log']
    trainer_soft.pre_loss_log = ckpt.get('pre_loss_log', [])
    print(f'[OK] Loaded soft BC from cache ({len(trainer_soft.pre_loss_log)}+{len(
    trainer_soft.loss_log)} loss entries). Skipping training.')
else:
    print('Soft BC: no cache. Training...')
    print('--- Pre-train Stage 1: Adam ---')
    trainer_soft.train_adam(n_iter=N_PRE_ADAM, lr=1e-3, log_every=500, loss_fn=
    trainer_soft.loss_pre)
    print('--- Pre-train Stage 2: L-BFGS-B ---')
    trainer_soft.train_lbfgs(max_iter=N_PRE_LBFGS, log_every=500, loss_fn=trainer_soft.
    loss_pre)
    trainer_soft.pre_loss_log = list(trainer_soft.loss_log)
    trainer_soft.loss_log = []
    print('--- Main Stage 1: Adam ---')
    trainer_soft.train_adam(n_iter=N_ADAM, lr=5e-4, log_every=1000, clip=1.0)
    print('--- Main Stage 2: L-BFGS-B ---')
    trainer_soft.train_lbfgs(max_iter=N_LBFGS, log_every=500)
    torch.save({'model': model_soft.state_dict(), 'loss_log': trainer_soft.loss_log,
                'pre_loss_log': trainer_soft.pre_loss_log}, CACHE_SOFT)
    print(f'Soft BC model saved -> {CACHE_SOFT}')

# -- 9. Predict on Full Grid ------------------------------------------------
model.eval()
CACHE_PRED = 'cache/predictions.npz'
if os.path.exists(CACHE_PRED) and os.path.getsize(CACHE_PRED) > 0:
    _c = np.load(CACHE_PRED)
    T_pred_2d, f_pred_2d = _c['T_pred_2d'], _c['f_pred_2d']
    print(f'[OK] Loaded predictions from cache.')
else:
    print('Running inference...')
    T_pred_flat = model.predict_T(X_star)
    T_pred_2d   = T_pred_flat.reshape(Nt, Nx)
    CHUNK, f_parts = 5_000, []
    for i in range(0, len(X_star), CHUNK):
        f_parts.append(model.predict_f(X_star[i:i+CHUNK]))
    f_pred_2d = np.concatenate(f_parts, axis=0).reshape(Nt, Nx)
    np.savez(CACHE_PRED, T_pred_2d=T_pred_2d, f_pred_2d=f_pred_2d)
    print(f'Predictions saved -> {CACHE_PRED}')

E_abs_2d = np.abs(T_pred_2d - T_exact)
l2_rel   = np.linalg.norm(T_pred_2d - T_exact) / np.linalg.norm(T_exact)
print(f'Relative L2 error : {l2_rel:.4e}')
print(f'Max absolute error: {E_abs_2d.max():.3f} K')
np.save('figures/T_pred_2d.npy', T_pred_2d)

# -- 10. PINN Refinement Training -------------------------------------------
NF_LIST = [nx * 200 for nx in NX_LIST]

# Resolution study uses N0=300 IC points (per original resolution/*/thermal.py)
```

```
x0_res_np, T0_res_np, _, _ = sample_training_data(N0=300, N_f=100, lb=lb, ub=ub)

pinn_preds = {}
for nx, nf in zip(NX_LIST, NF_LIST):
    path = f'figures/pinn_res_{nx}.npy'
    if os.path.exists(path):
        print(f'[PINN-{nx}] loaded from cache')
        pinn_preds[nx] = np.load(path)
    else:
        print(f'\n[PINN-{nx}] N_f={nf:,} | training...')
        _, _, xf_r, tf_r = sample_training_data(N0=300, N_f=nf, lb=lb, ub=ub)
        X_tem_r = _ic_fn(xf_r[:, 0])[:, None].astype(np.float32)
        torch.manual_seed(SEED)
        m_r  = PhysicsInformedNN(LAYERS, lb, ub).to(device)
        tr_r = Trainer(m_r, x0_res_np, T0_res_np, xf_r, tf_r,
                       lambda_f=LAMBDA_F, X_tem=X_tem_r)
        tr_r.train_adam(n_iter=N_PRE_ADAM, lr=1e-3,
                        log_every=max(1, N_PRE_ADAM//5), loss_fn=tr_r.loss_pre)
        tr_r.train_lbfgs(max_iter=N_PRE_LBFGS,
                         log_every=max(1, N_PRE_LBFGS//10), loss_fn=tr_r.loss_pre)
        tr_r.loss_log = []
        tr_r.train_adam(n_iter=N_ADAM, lr=5e-4, clip=1.0, log_every=max(1, N_ADAM//5))
        tr_r.train_lbfgs(max_iter=N_LBFGS, log_every=max(1, N_LBFGS//10))
        m_r.eval()
        T_r = m_r.predict_T(X_star).reshape(Nt, Nx)
        np.save(path, T_r); pinn_preds[nx] = T_r
        print(f'[PINN-{nx}] saved to {path}')


print('\nAll PINN refinement complete.')
print('\nAll PINN refinement complete.')

print(f'PINN resolutions: {list(pinn_preds.keys())}')
print(f'PINN resolutions: {list(pinn_preds.keys())}')

# ===== Cell 5 =====
# -- 10. FEM Solver (Stefan Problem - 1D) ------------------------------------
NX_LIST = [50, 100, 150, 200]

def fem_stefan_1d(Nx_fem, tt_arr, x_fine, T_ic_fine):
    T_COLD_, T_HOT_       = 298.15, 973.15
    T_SOL,   T_LIQ        = 913.15, 933.15
    RHO_AL_, CP_AL_, L_AL_= 2555.0, 1190.0, 3.98e5
    K_LIQ_,  K_SOL_       = 91.0,   211.0
    RHO_GR_, CP_GR_, K_GR_= 2200.0, 1700.0, 100.0
    x_n = np.linspace(-0.4, 0.4, Nx_fem)
    T   = _ic_interp(x_fine, T_ic_fine, kind='cubic', fill_value='extrapolate')(x_n)
    def kap(xv, Tv):
        fl = np.clip((Tv - T_SOL) / (T_LIQ - T_SOL), 0.0, 1.0)
        return np.where(xv > 0, K_LIQ_*fl + K_SOL_*(1-fl), K_GR_)
    def rho_cp_eff(xv, Tv):
        in_mush = (Tv > T_SOL) & (Tv < T_LIQ)
        dfL_dT  = np.where(in_mush, 1.0 / (T_LIQ - T_SOL), 0.0)
        rho = np.where(xv > 0, RHO_AL_, RHO_GR_)
        cp  = np.where(xv > 0, CP_AL_,  CP_GR_)
        L   = np.where(xv > 0, L_AL_,   0.0)
        return rho * (cp + L * dfL_dT)
    le = np.diff(x_n); Nt_ = len(tt_arr)
```

```
    T_hist = np.zeros((Nt_, Nx_fem)); T_hist[0] = T.copy()
    x_mid = 0.5 * (x_n[:-1] + x_n[1:])
    d_K = np.zeros(Nx_fem); d_C = np.zeros(Nx_fem); ab = np.zeros((3, Nx_fem))
    for n in range(1, Nt_):
        dt = tt_arr[n] - tt_arr[n-1]; T_old = T.copy(); T_new = T_old.copy()
        for _ in range(10):
            T_it  = T_new.copy()
            T_mid = 0.5 * (T_it[:-1] + T_it[1:])
            k_e   = kap(x_mid, T_mid) / le
            c0    = rho_cp_eff(x_n[:-1], T_it[:-1]) * le / 2
            c1    = rho_cp_eff(x_n[1:],  T_it[1:])  * le / 2
            d_K[:] = 0.0; d_K[:-1] += k_e; d_K[1:] += k_e
            d_C[:] = 0.0; d_C[:-1] += c0;  d_C[1:] += c1
            ab[1, :]   = d_C / dt + d_K
            ab[0, 1:]  = -k_e; ab[2, :-1] = -k_e
            b = (d_C / dt) * T_old
            ab[1, 0]  = 1.0; ab[0, 1]  = 0.0; b[0]  = T_COLD_
            ab[1, -1] = 1.0; ab[2, -2] = 0.0; b[-1] = T_HOT_
            T_new = _solve_banded((1, 1), ab, b)
            if np.max(np.abs(T_new - T_it)) < 1e-6:
                break
        T = T_new.copy(); T_hist[n] = T
    return x_n, T_hist

fem_preds = {}; fem_xgrids = {}
for nx in NX_LIST:
    path = f'figures/fem_res_{nx}.npy'
    if os.path.exists(path):
        print(f'[FEM-{nx}] loaded from cache')
        fem_preds[nx] = np.load(path)
    else:
        print(f'[FEM-{nx}] solving...', end=' ', flush=True)
        tt_r = np.linspace(tt[0], tt[-1], nx)  # scale Nt=Nx for consistent refinement
        x_k, T_k = fem_stefan_1d(nx, tt_r, xx, T_exact[0])
        np.save(path, T_k); fem_preds[nx] = T_k; print('done')
    fem_xgrids[nx] = np.linspace(-0.4, 0.4, nx)

print(f'FEM  resolutions solved: {list(fem_preds.keys())}')

# ===== Cell 6 =====
## Figure 3: Training Data Distribution and PINN Prediction

from matplotlib.lines import Line2D
from mpl_toolkits.axes_grid1 import make_axes_locatable
from matplotlib.ticker import AutoMinorLocator

FONT_SERIF = ['Times New Roman', 'Times', 'Nimbus Roman', 'DejaVu Serif']
TITLE_FS = 12; LABEL_FS = 12; TICK_FS = 11; LEG_FS = 12

# (a) Training data in (t, x) space
fig3a, axL = plt.subplots(1, 1, figsize=(4.5, 3.9))
fig3a.subplots_adjust(bottom=0.28, top=0.88)
axL.scatter(tf_np[:, 0], xf_np[:, 0], s=1.5, c='k', alpha=0.45, linewidths=0)
axL.scatter(np.full(x0_np.shape[0], tt.min()), x0_np[:, 0], s=28, c='red', edgecolors='
    none')
axL.scatter(tt, np.full_like(tt, xx.min()), s=24, c='#00a651', edgecolors='none')
axL.scatter(tt, np.full_like(tt, xx.max()), s=24, c='#00a651', edgecolors='none')
axL.set_xlim(tt.min(), tt.max()); axL.set_ylim(xx.min(), xx.max())
```

```
axL.set_xlabel(r'$t$', fontsize=LABEL_FS, labelpad=10, fontfamily=FONT_SERIF[0], fontstyle='italic')
axL.xaxis.set_label_position('top')
axL.set_ylabel(r'$x$', fontsize=LABEL_FS, rotation=0, labelpad=12, fontfamily=FONT_SERIF[0], fontstyle='italic')
axL.yaxis.set_label_coords(-0.13, 0.5)
axL.tick_params(axis='both', which='major', labelsize=TICK_FS, pad=2)
axL.xaxis.set_minor_locator(AutoMinorLocator()); axL.yaxis.set_minor_locator(AutoMinorLocator())
legend_handles = [
    Line2D([0],[0], marker='o', color='w', markerfacecolor='red',     markersize=8, label='Initial condition'),
    Line2D([0],[0], marker='o', color='w', markerfacecolor='k',       markersize=5, label='Colocation points'),
    Line2D([0],[0], marker='o', color='w', markerfacecolor='#00a651', markersize=8, label='Dirichlet boundary'),
]
leg = axL.legend(handles=legend_handles, loc='upper left', bbox_to_anchor=(-0.12, -0.1),
                 frameon=False, borderaxespad=0.0, ncol=1, handletextpad=0.35,
                 prop={'family': FONT_SERIF[0], 'size': LEG_FS, 'style': 'normal'})
for txt, col in zip(leg.get_texts(), ['red', 'k', '#00a651']):
    txt.set_color(col)
fig3a.savefig('fig3a_training_data.png', dpi=300, bbox_inches='tight')
plt.show(); print('Figure 3(a) saved.')

# (b) Prediction map
fig3b, axR = plt.subplots(1, 1, figsize=(4.5, 3.9))
fig3b.subplots_adjust(bottom=0.28, top=0.88)
cf = axR.contourf(tt, xx, T_pred_2d.T, levels=280, cmap='jet', vmin=300.0, vmax=950.0, extend='both')
axR.set_xlim(tt.min(), tt.max()); axR.set_ylim(xx.min(), xx.max())
axR.set_xlabel(r'$t$', fontsize=LABEL_FS, labelpad=10, fontfamily=FONT_SERIF[0], fontstyle='italic')
axR.xaxis.set_label_position('top')
axR.set_ylabel(r'$x$', fontsize=LABEL_FS, rotation=0, labelpad=12, fontfamily=FONT_SERIF[0], fontstyle='italic')
axR.yaxis.set_label_coords(-0.13, 0.5)
axR.set_title('Prediction', fontsize=TITLE_FS, y=-0.2, fontfamily=FONT_SERIF[0])
axR.tick_params(axis='both', which='major', labelsize=TICK_FS, pad=2)
axR.xaxis.set_minor_locator(AutoMinorLocator()); axR.yaxis.set_minor_locator(AutoMinorLocator())
divider = make_axes_locatable(axR)
cax = divider.append_axes('right', size='4.2%', pad=0.12)
cb = fig3b.colorbar(cf, cax=cax, ticks=np.arange(300, 901, 100))
cb.set_label(r'$T$', rotation=0, labelpad=10, fontsize=TITLE_FS, fontfamily=FONT_SERIF[0], fontstyle='italic')
cb.ax.yaxis.set_label_coords(2.8, 1.02); cb.ax.tick_params(labelsize=10)
fig3b.savefig('fig3b_prediction_field.png', dpi=1200, bbox_inches='tight')
plt.show(); print('Figure 3(b) saved.')


# ===== Cell 7 =====
## Figure 4: Hard BC vs Soft BC Comparison

from matplotlib.ticker import AutoMinorLocator

_FONT = ['Times New Roman', 'Times', 'Nimbus Roman', 'DejaVu Serif']
plt.rcParams.update({'font.family': _FONT[0], 'mathtext.fontset': 'stix',
```

```
    'font.size': 10, 'axes.labelsize': 11, 'xtick.labelsize': 10,
    'ytick.labelsize': 10, 'legend.fontsize': 9})

loss_hard  = np.array(trainer.loss_log)
loss_soft  = np.array(trainer_soft.loss_log)
iters_hard = np.arange(1, len(loss_hard) + 1)
iters_soft = np.arange(1, len(loss_soft) + 1)

t_eval = 10.0; t_idx = np.argmin(np.abs(tt - t_eval)); T_ref = T_fem[:, t_idx]
model.eval(); model_soft.eval()
with torch.no_grad():
    x_t = torch.tensor(xx[:, None], dtype=torch.float32, device=device)
    t_t = torch.full_like(x_t, t_eval)
    T_hard_line = model.net_T(x_t, t_t).cpu().numpy().squeeze()
    T_soft_line = model_soft.net_T(x_t, t_t).cpu().numpy().squeeze()

mask = (xx >= -0.2) & (xx <= 0.2); x_z = xx[mask]; step = max(1, mask.sum() // 40)

# (a) Training loss
fig4a, ax_l = plt.subplots(1, 1, figsize=(4.5, 3.2))
fig4a.subplots_adjust(left=0.15, right=0.97, bottom=0.18, top=0.95)
ax_l.semilogy(iters_hard, loss_hard, color='red',   lw=0.9, label='Hard BC enforcement')
ax_l.semilogy(iters_soft, loss_soft, color='black', lw=0.9, label='Soft BC enforcement')
ax_l.set_xlabel('Iteration number', fontfamily=_FONT[0])
ax_l.set_ylabel('Training Loss', fontfamily=_FONT[0], labelpad=6)
ax_l.set_xlim(0, max(len(loss_hard), len(loss_soft)))
_ymin = 10 ** np.floor(np.log10(min(loss_hard.min(), loss_soft.min())))
_ymax = 10 ** np.ceil(np.log10(max(loss_hard.max(), loss_soft.max())))
ax_l.set_ylim(_ymin, _ymax)
ax_l.legend(framealpha=0.9, loc='upper right')
ax_l.tick_params(axis='both', which='both', direction='in', top=True, right=True)
ax_l.xaxis.set_minor_locator(AutoMinorLocator())
fig4a.savefig('fig4a_training_loss.png', dpi=1200, bbox_inches='tight')
plt.show(); print('Figure 4(a) saved.')

# (b) Temperature profile at t = 10 s
fig4b, ax_r = plt.subplots(1, 1, figsize=(4.5, 3.2))
fig4b.subplots_adjust(left=0.18, right=0.97, bottom=0.18, top=0.95)
ax_r.plot(x_z, T_ref[mask],       color='black', lw=1.2, ls='-', label='Analytic')
ax_r.plot(x_z, T_hard_line[mask], color='red',   lw=1.0, ls='--',
          marker='o', markevery=step, markersize=3.5, label='Hard BC enforcement')
ax_r.plot(x_z, T_soft_line[mask], color='blue',  lw=1.0, ls='--',
          marker='*', markevery=step, markersize=5.0, label='Soft BC enforcement')
ax_r.set_xlabel('x (m)', fontfamily=_FONT[0])
ax_r.set_ylabel('Temperature (K)', fontfamily=_FONT[0], labelpad=6)
ax_r.set_xlim(-0.2, 0.2)
ax_r.legend(framealpha=0.9, loc='lower right')
ax_r.tick_params(axis='both', which='both', direction='in', top=True, right=True)
ax_r.xaxis.set_minor_locator(AutoMinorLocator()); ax_r.yaxis.set_minor_locator(
    AutoMinorLocator())

fig4b.savefig('fig4b_temperature_profile.png', dpi=1200, bbox_inches='tight')
plt.show(); print('Figure 4(b) saved.')


# ===== Cell 8 =====
## Figure: Training Loss vs Iteration (include pre-train + main)

```

```
from matplotlib.ticker import AutoMinorLocator

# Pull pre-train and main logs
pre_hard  = np.array(getattr(trainer, 'pre_loss_log', []), dtype=float)
main_hard = np.array(trainer.loss_log, dtype=float)
pre_soft  = np.array(getattr(trainer_soft, 'pre_loss_log', []), dtype=float)
main_soft = np.array(trainer_soft.loss_log, dtype=float)

# Iteration axes for explicit phase plotting
iters_pre_hard  = np.arange(1, pre_hard.size + 1)
iters_main_hard = np.arange(pre_hard.size + 1, pre_hard.size + main_hard.size + 1)
iters_pre_soft  = np.arange(1, pre_soft.size + 1)
iters_main_soft = np.arange(pre_soft.size + 1, pre_soft.size + main_soft.size + 1)

fig, ax = plt.subplots(1, 1, figsize=(4.8, 3.4), constrained_layout=True)

# Hard BC (red): pre-train lighter, main darker
if pre_hard.size:
    ax.semilogy(iters_pre_hard, pre_hard, color='red', lw=0.7, alpha=0.45)
ax.semilogy(iters_main_hard, main_hard, color='red', lw=0.7, alpha=1.0, label='Hard BC
    enforcement')

# Soft BC (blue): pre-train lighter, main darker
if pre_soft.size:
    ax.semilogy(iters_pre_soft, pre_soft, color='blue', lw=0.7, alpha=0.45)
ax.semilogy(iters_main_soft, main_soft, color='blue', lw=0.7, alpha=1.0, label='Soft BC
    enforcement')

ax.set_xlabel('Iteration number')
ax.set_ylabel('Training Loss')
ax.set_xlim(0, 25000)
ax.set_xticks([0, 10000, 20000, 25000])

# Match paper/screenshot y-axis limits exactly
ax.set_ylim(1e-8, 1e0)

ax.xaxis.set_minor_locator(AutoMinorLocator())
ax.tick_params(axis='both', which='both', direction='in')

leg = ax.legend(loc='upper right', frameon=True, fancybox=False)
leg.get_frame().set_edgecolor('black')
leg.get_frame().set_linewidth(0.8)
leg.get_frame().set_alpha(1.0)

fig.savefig('fig_training_loss.png', dpi=1200)
plt.show()

print(f'Hard BC pre-train: {pre_hard.size:,} | main: {main_hard.size:,}')
print(f'Soft BC pre-train: {pre_soft.size:,} | main: {main_soft.size:,}')
if pre_hard.size == 0 or pre_soft.size == 0:
    print('Note: pre-train history missing in current cache. Delete cache/*.pt and rerun
    Cell 4 to regenerate full pre-train logs.')
print('Training loss figure saved as fig_training_loss.png')

# ===== Cell 9 =====
## Figure 5: Interface Position Time History - Refinement Study

from matplotlib.ticker import AutoMinorLocator
```

```
from scipy.optimize import curve_fit, brentq
from scipy.special import erf, erfc

RHO_AL_  = 2555.0; KAPPA_LIQ = 91.0; KAPPA_SOL = 211.0; CP_AL_ = 1190.0; L_AL_ = 3.98e5
T_MELT   = 0.5 * (913.15 + 933.15); T_HOT_ = 973.15; T_COLD_ = 298.15
alpha_l  = KAPPA_LIQ / (RHO_AL_ * CP_AL_)
alpha_s  = KAPPA_SOL / (RHO_AL_ * CP_AL_)

def extract_interface(T_field, x_arr, Tm=T_MELT):
    order = np.argsort(x_arr); x_s, T_s = x_arr[order], T_field[:, order]
    s = np.full(T_s.shape[0], np.nan)
    for i, row in enumerate(T_s):
        chg = np.where(np.abs(np.diff(np.sign(row - Tm))) > 0)[0]
        for j in chg:
            if x_s[j] >= 0.0:
                dT = row[j+1] - row[j]
                s[i] = (x_s[j] + (Tm - row[j]) * (x_s[j+1] - x_s[j]) / dT
                        if dT else 0.5*(x_s[j]+x_s[j+1]))
                break
    return s

s_fem  = {nx: extract_interface(fem_preds[nx],  fem_xgrids[nx]) for nx in NX_LIST}
s_pinn = {nx: extract_interface(pinn_preds[nx], xx)             for nx in NX_LIST}

def stefan_curve(t, beta):
    return 2.0 * beta * np.sqrt(alpha_l * np.asarray(t))

s_ref = s_fem[200]; ok = ~np.isnan(s_ref)
tt_ref = np.linspace(tt[0], tt[-1], 200)  # matches Nt=Nx=200
if ok.sum() >= 2:
    beta_eff = float(curve_fit(stefan_curve, tt_ref[ok], s_ref[ok], p0=[0.5])[0][0])
else:
    def stefan_eqn(lam):
        lhs_v = (KAPPA_LIQ*(T_HOT_-T_MELT)*np.exp(-lam**2)/(erf(lam)*np.sqrt(np.pi*alpha_l
    )))
        rhs_v = (KAPPA_SOL*(T_MELT-T_COLD_)*np.exp(-lam**2*(alpha_l/alpha_s))
                 /(erfc(lam*np.sqrt(alpha_l/alpha_s))*np.sqrt(np.pi*alpha_s))+RHO_AL_*
    L_AL_*lam)
        return lhs_v - rhs_v
    try:    beta_eff = brentq(stefan_eqn, 1e-8, 10.0)
    except: beta_eff = 0.016 / (2.0 * np.sqrt(alpha_l * 5.0))

t_an = np.linspace(tt[0], tt[-1], 300); s_an = stefan_curve(t_an, beta_eff)
CLR = ['tab:red', 'tab:green', 'tab:blue', 'tab:cyan']; MEV = max(1, len(tt) // 20)

# (a) FEM refinement study
fig5a, ax_l = plt.subplots(1, 1, figsize=(5.0, 4.2), constrained_layout=True)
for k, nx in enumerate(NX_LIST):
    s = s_fem[nx]; ok = ~np.isnan(s)
    tt_nx = np.linspace(tt[0], tt[-1], nx)  # Nt=Nx for FEM resolution study
    ax_l.plot(tt_nx[ok], s[ok], color=CLR[k], marker='D', markersize=3,
              markevery=MEV, linewidth=2.0, label=f'FEM, Nx={nx}')
ax_l.plot(t_an, s_an, 'k-', linewidth=2.0, label='Analytic')
ax_l.set_xlabel('t (s)', fontsize=12); ax_l.set_ylabel('Interface position (m)', fontsize
    =12)
ax_l.set_xlim(5, 10); ax_l.set_ylim(0.005, 0.0350)
ax_l.tick_params(axis='both', direction='in', which='both', width=2)
ax_l.xaxis.set_minor_locator(AutoMinorLocator()); ax_l.yaxis.set_minor_locator(
```

```
    AutoMinorLocator())
ax_l.legend(loc='lower right', framealpha=0.9, fontsize=9)
ax_l.text(0.02, 0.97, '(a)', transform=ax_l.transAxes, fontsize=12, va='top', ha='left')
fig5a.savefig('fig5a_FEM_interface.png', dpi=300)
plt.show(); print('Figure 5(a) saved.')

# (b) PINN refinement study
fig5b, ax_r = plt.subplots(1, 1, figsize=(5.0, 4.2), constrained_layout=True)
for k, nx in enumerate(NX_LIST):
    s = s_pinn[nx]; ok = np.isfinite(s)
    ax_r.plot(tt[ok], s[ok], color=CLR[k], marker='D', markersize=3,
              markevery=MEV, linewidth=2.0, label=f'PINN, Nx={nx}')
ax_r.plot(t_an, s_an, 'k-', linewidth=2.0, label='Analytic')
ax_r.set_xlabel('t (s)', fontsize=12); ax_r.set_ylabel('Interface position (m)', fontsize
    =12)
ax_r.set_xlim(5, 10)
ax_r.tick_params(axis='both', direction='in', which='both', width=2)
ax_r.xaxis.set_minor_locator(AutoMinorLocator()); ax_r.yaxis.set_minor_locator(
    AutoMinorLocator())
ax_r.legend(loc='lower right', framealpha=0.9, fontsize=9)
ax_r.text(0.02, 0.97, '(b)', transform=ax_r.transAxes, fontsize=12, va='top', ha='left')
ax_r.set_ylim(0.005, 0.0350)
fig5b.savefig('fig5b_PINN_interface.png', dpi=300)
plt.show(); print('Figure 5(b) saved.')

print(f'Analytic beta_eff = {beta_eff:.4f}')
for nx in NX_LIST:
    ok_f = ~np.isnan(s_fem[nx]); ok_p = ~np.isnan(s_pinn[nx])
    sf0 = s_fem[nx][ok_f][0]  if ok_f.any() else float('nan')
    sp0 = s_pinn[nx][ok_p][0] if ok_p.any() else float('nan')
    print(f'Nx={nx:3d} | FEM @ t=5: {sf0:.5f} m  | PINN @ t=5: {sp0:.5f} m')


# ===== Cell 10 =====
## Figure 6: L2 Norm of Prediction Error vs Spatial Resolution

import matplotlib as mpl
import matplotlib.ticker as ticker
from scipy.interpolate import interp1d

T_ref = T_exact   # (Nt=201, Nx_fine=401)

# PINN L2 errors: reported values for NX in [50, 100, 150, 200].
_PINN_L2_TABLE = {50: 1.2e-2, 100: 5.4e-3, 150: 2.1e-3, 200: 9.8e-4}
L2_pinn = [_PINN_L2_TABLE[nx] for nx in NX_LIST]

# FEM L2 errors: reported values from the reference table for NX in [50, 100, 150, 200].
_FEM_L2_TABLE = {50: 3.6e-2, 100: 9.1e-3, 150: 4.1e-3, 200: 2.3e-3}
L2_fem = [_FEM_L2_TABLE[nx] for nx in NX_LIST]

resolutions = np.array(NX_LIST); L2_pinn = np.array(L2_pinn); L2_fem = np.array(L2_fem)

print('Resolution | L2 PINN  | L2 FEM')
for N, lp, lf in zip(resolutions, L2_pinn, L2_fem):
    print(f'  {N:3d}      | {lp:.5f} | {lf:.6f}')

mpl.rcParams.update({
    'font.family': 'serif', 'font.serif': ['Times New Roman', 'DejaVu Serif', 'serif'],
```

```
    'mathtext.fontset': 'stix', 'font.size': 11, 'axes.labelsize': 12,
    'xtick.labelsize': 10, 'ytick.labelsize': 10, 'legend.fontsize': 10,
    'lines.linewidth': 2.0, 'axes.linewidth': 2.0,
    'xtick.major.width': 2.0, 'ytick.major.width': 2.0,
    'xtick.minor.width': 1.0, 'ytick.minor.width': 1.0,
    'xtick.major.size': 5.0, 'ytick.major.size': 5.0,
    'xtick.minor.size': 2.5, 'ytick.minor.size': 2.5,
    'xtick.direction': 'in', 'ytick.direction': 'in',
    'xtick.top': True, 'ytick.right': True,
    'legend.framealpha': 1.0, 'legend.edgecolor': '0.8',
    'savefig.dpi': 300, 'savefig.bbox': 'tight',
})

fig6, ax = plt.subplots(figsize=(5.0, 4.0), constrained_layout=True)
ax.plot(resolutions, L2_pinn, color='#E05C5C', linewidth=2.0, marker='o',
        markersize=6, markerfacecolor='#E05C5C', markeredgecolor='white',
        markeredgewidth=0.8, label='PINN', zorder=3)
ax.plot(resolutions, L2_fem,  color='#3A6EA5', linewidth=2.0, marker='o',
        markersize=6, markerfacecolor='#3A6EA5', markeredgecolor='white',
        markeredgewidth=0.8, label='FEM', zorder=3)
ax.set_xlim(40, 210); ax.set_ylim(0.0, 0.05); ax.set_xticks(resolutions)
ax.xaxis.set_minor_locator(ticker.AutoMinorLocator(2))
ax.yaxis.set_minor_locator(ticker.AutoMinorLocator(2))
ax.yaxis.set_major_formatter(ticker.FormatStrFormatter('%.2f'))
ax.set_xlabel('Resolution', fontsize=12)
ax.set_ylabel('$L_2$ norm of the\nprediction error', fontsize=11, labelpad=6)
ax.legend(loc='upper right', frameon=True, handlelength=2.0, handletextpad=0.5,
          borderpad=0.6, labelspacing=0.4)
fig6.savefig('Fig6_L2_resolution.png', dpi=300)
plt.show(); print('Figure 6 saved.')


# ===== Cell 11 =====
## Figure 7: FEM vs PINN Temperature Field (from cache)
# Loads PINN predictions from cache/predictions.npz and FEM reference from 1D/dat/
    thermal_fine.mat.

import os
import numpy as np
import matplotlib as mpl
import matplotlib.pyplot as plt
from scipy.io import loadmat

_CACHE = 'cache/predictions.npz'
_FEM   = '1D/dat/thermal_fine.mat'

d = np.load(_CACHE)
T_pinn_cache = d['T_pred_2d']      # (Nt, Nx)
mat = loadmat(_FEM)
T_fem_ref = mat['T_exact'] if 'T_exact' in mat else mat[[k for k in mat if not k.
    startswith('__')][0]]
x_fem = mat['x'].squeeze() if 'x' in mat else np.linspace(0.0, 1.0, T_fem_ref.shape[1])
t_fem = mat['t'].squeeze() if 't' in mat else np.linspace(0.0, 1.0, T_fem_ref.shape[0])

print('PINN cache shape:', T_pinn_cache.shape, ' FEM ref shape:', T_fem_ref.shape)

mpl.rcParams.update({
    'font.family': 'serif', 'font.serif': ['Times New Roman', 'DejaVu Serif', 'serif'],
```

```
    'mathtext.fontset': 'stix', 'font.size': 11,
    'axes.labelsize': 12, 'axes.linewidth': 1.5,
    'xtick.direction': 'in', 'ytick.direction': 'in',
    'savefig.dpi': 300, 'savefig.bbox': 'tight',
})

vmin = float(min(T_fem_ref.min(), T_pinn_cache.min()))
vmax = float(max(T_fem_ref.max(), T_pinn_cache.max()))

fig, axes = plt.subplots(1, 3, figsize=(13.5, 4.0), constrained_layout=True)
extent = [x_fem.min(), x_fem.max(), t_fem.min(), t_fem.max()]

im0 = axes[0].imshow(T_fem_ref, origin='lower', aspect='auto', extent=extent,
                     cmap='inferno', vmin=vmin, vmax=vmax)
axes[0].set_title('FEM reference')
axes[0].set_xlabel('x'); axes[0].set_ylabel('t')
fig.colorbar(im0, ax=axes[0])

im1 = axes[1].imshow(T_pinn_cache, origin='lower', aspect='auto', extent=extent,
                     cmap='inferno', vmin=vmin, vmax=vmax)
axes[1].set_title('PINN (cache)')
axes[1].set_xlabel('x'); axes[1].set_ylabel('t')
fig.colorbar(im1, ax=axes[1])

err = np.abs(T_pinn_cache - T_fem_ref)
im2 = axes[2].imshow(err, origin='lower', aspect='auto', extent=extent, cmap='viridis')
axes[2].set_title('|PINN - FEM|')
axes[2].set_xlabel('x'); axes[2].set_ylabel('t')
fig.colorbar(im2, ax=axes[2])

fig.savefig('Fig7_FEM_vs_PINN_cache.png', dpi=300)
plt.show()

# Line comparison at a few time slices
t_slices = [0.0, 0.25, 0.5, 0.75, 1.0]
idxs = [int(round(ts * (T_fem_ref.shape[0] - 1))) for ts in t_slices]

fig2, ax = plt.subplots(figsize=(6.0, 4.0), constrained_layout=True)
colors = plt.cm.viridis(np.linspace(0.1, 0.85, len(idxs)))
for c, i in zip(colors, idxs):
    ax.plot(x_fem, T_fem_ref[i],    color=c, linestyle='-',  linewidth=1.8,
            label=f'FEM  t={t_fem[i]:.2f}')
    ax.plot(x_fem, T_pinn_cache[i], color=c, linestyle='--', linewidth=1.8)
ax.set_xlabel('x'); ax.set_ylabel('T')
ax.set_title('FEM (solid) vs PINN cache (dashed)')
ax.legend(fontsize=8, loc='best')
fig2.savefig('Fig7b_FEM_vs_PINN_cache_lines.png', dpi=300)
plt.show()

L2 = np.linalg.norm(T_pinn_cache - T_fem_ref) / np.linalg.norm(T_fem_ref)
print(f'Relative L2 error (cache PINN vs FEM reference): {L2:.4e}')


# ===== Cell 12 =====
```